

Self Consistent Image Force Interaction + virtual AFM machine

Lev Kantorovich, Thomas Trevethan*, Jerome Polesel-Maris[†],
Natalia Martsinovich[‡] and Adam Foster[§]

February 12, 2009

Physics, King's College London, The Strand, London, WC2R 2LS, United Kingdom

Contents

I	Theoretical Models	5
1	Semi-classical part (SciFi)	5
1.1	Introduction	5
1.2	Electrostatic energy of a system of metals and charges	6
1.3	Tip-surface Interaction	7
1.4	Solution of the electrostatic problem of point charges inside the sphere-plane capacitor	8
1.5	The calculation of the total force acting on the tip	10
1.5.1	Definition of the Instantaneous Force on the Tip (for MD)	10
1.6	Interatomic Potentials	11
1.6.1	Pairwise interaction	12
1.6.2	Many-body interactions	12
1.6.3	Tersoff interactions	13
1.7	Constrained minimisation	14
1.7.1	How constraints are dealt with in the code	14
1.7.2	Linear constraints	15
1.7.3	Constant distance between two atoms	15
1.7.4	Constant angle between three atoms	16
1.8	Molecular Dynamics Algorithms	16
1.8.1	NVE Verlet Leapfrog	16
1.8.2	NVT with Gaussian Thermostat	16
1.8.3	NVT with Berendsen Thermostat	17
1.8.4	NVT with Nose-Hoover Thermostat	17
1.8.5	Stochastic Boundary Conditions	17
1.8.6	Shells	17
2	Virtual AFM	18
2.1	How the virtual machine works	18
2.1.1	Equation of motion of the cantilever	18
2.1.2	Amplitude regulation	19
2.1.3	Frequency demodulation and cantilever excitation	20
2.1.4	Distance regulation	21

*University College London, U.K.

[†]CEMES-CNRS, Toulouse, France

[‡]Warwick University, U.K.

[§]University of Helsinki, Finland

2.2	Influence of experimental noises on AFM images	21
2.3	How one can find parameters of the virtual AFM	22
II	Installation	23
3	How to compile the code	23
4	parameter file	23
5	virtual_param.inc file	24
6	How to run the code	24
III	Input	25
7	General	25
8	SciFi input	25
8.1	Example Input File	25
8.2	Tolerance Box	27
8.2.1	General settings	27
8.2.2	Output control	28
8.2.3	Geometry optimisation options	29
8.2.4	Testing of the code	30
8.3	Species	30
8.3.1	ShellPar Box	30
8.3.2	Species_Types	31
8.4	Box with atomic coordinates	31
8.4.1	move	31
8.4.2	rotate	31
8.4.3	centre	31
8.4.4	scan	32
8.4.5	Atomic Coordinates	32
8.5	Interactions between atoms	32
8.5.1	PairPotenc Box	32
8.5.2	BondTolerance Box	33
8.5.3	quadratic_bond Box	33
8.5.4	quadratic_angle Box	33
8.5.5	torsion_1 Box	33
8.5.6	out-of-plane Box	34
8.5.7	Tersof Box	34
8.6	Lattice vectors	35
8.7	Molecular Dynamics boxes	35
8.7.1	MDcontrol Box	35
8.7.2	Velocity Box	37
8.7.3	Friction Box	37
8.8	Dynamic Tip	37
8.8.1	During MD	37
8.8.2	During energy relaxation	38
8.9	Image Box	38
8.9.1	plane	38
8.9.2	Z_plane	38
8.9.3	sphere	38
8.9.4	bottom_sph	38
8.9.5	radius	39
8.9.6	bias	39

8.9.7	qwrite	39
8.9.8	printing	39
8.9.9	precis	39
8.9.10	decay	39
8.9.11	smooth	39
8.9.12	buffer	40
8.10	Constraints Box	40
8.11	Quantum cluster	40
9	vAFM input	41
9.1	Example Input File	41
9.2	Virtual AFM Box	42
9.2.1	General settings for the cantilever	42
9.2.2	Default set points	43
9.2.3	Initial position of the tip	44
9.2.4	The frequency of the general printout	44
9.2.5	Parameters for the Automatic Gain Control (AGC)	44
9.2.6	Parameters for the Phase Lock Loop (PLL)	44
9.2.7	Parameters for the Automatic Distance Control (ADC)	45
9.2.8	Times	45
9.3	Force Field Box	45
9.3.1	How to specify the short-range force	47
9.3.2	Long-range part of the force	47
9.3.3	*Parameters necessary to run vAFM and SciFi together	48
9.3.4	Testing the whole range force field	48
9.3.5	Structure of the force-field file	49
9.4	Elementary Stages Box	49
9.4.1	What controls every elementary stage?	49
9.4.2	Specifying each elementary stage	50
9.4.3	Doing a 2D scan	51
IV	Output	53
10	SciFi specific output	53
10.1	Standard Output	53
10.2	Output File	53
10.3	Configuration Files	53
10.4	Statistics File	53
10.5	Trajectory File	53
10.6	Tip Force Files	53
10.7	Output from image	54
11	vAFM specific output	54
11.1	virtual.out	54
11.2	stageM.out	54
11.3	signals.dat	54
11.4	2Dimage_S.dat	55
11.5	ftip.dat	55
V	Example Inputs	56

Introduction

Motivated by the need to aid in the interpretation of experimental Atomic Force Microscopy (AFM) images, this code was developed to self-consistently calculate the interaction between a tip and surface, and then use this interaction in running a virtual AFM machine that simulates a real experimental device. This may be time consuming. However, we believe this must be the option of choice in simulations that involve heavily the response of the probe on the surface species, e.g. during *manipulation* experiments. As far as we are aware, this is the most advanced AFM simulating tool today.

The code can be run in three regimes:

- as a semi-classical simulation tool (known separately as the SciFi code)
- as a virtual AFM machine (vAFM), in which case a 3D force field is assumed to be prepared separately (preprocessed); in this case the SciFi part of the code is not activated;
- as a combination of the two. In this case the vAFM machine is used as an engine for moving the tip. At each tip position the atoms in the microscopic part are allowed to relax to mechanical equilibrium using the specified interactions between them, and then the force on the tip is calculated. This is performed by the SciFi part of the code. The tip force is then fed into the vAFM part to process, and, by advancing difference equations governing electronics, a new position of the tip is calculated that is fed back into the SciFi part.

The Sci-Fi part of the code has the facility to perform many different types of calculations on a tip-surface set up. The program can either be used to find the minimum energy configuration of a given system using powerful minimisation algorithms, or perform a finite temperature Molecular Dynamics (MD) simulation in a number of different ensembles.

Microscopic systems can be composed of a variety of different classical potentials and the program has the facility to implement atomic polarisation via the shell model, where pairwise interactions are modeled by a series of Buckingham style potentials. The modeling of covalent crystals and organic molecules is possible by the use of many-body potentials which comprise of bond, angle, torsion and out-of-plane interactions. Some (or all) can also interact via Tersoff-like potential. All types of interaction (pairwise, manybody and Tersoff) can be combined, i.e. all atoms of the system can be split into a set of Tersoff-like atoms, many-body atoms and the rest which interact between themselves and with the Tersoff and manybody atoms via a pairwise interaction.

The AFM functionality is facilitated via a number of ways. In the simplest ones, a user can simply specify how the tip moves (e.g. vertical oscillation or a linear displacement along a certain direction). The tip is moved in small steps, and at each step the atomic system is either allowed to relax or moves according to Newtonian equations of motion, i.e. a MD calculation is performed (a non-equilibrium MD).

A new powerful feature, a vAFM, has been recently added. It is designed for simulating the work of electronics of an actual Non-Contact AFM device. If this functionality is on, then the movement of the tip is controlled by equations simulating electronics of an actual AFM apparatus. In this case, the tip can be moved in a number of ways, e.g. by performing a 2D scan.

The calculation resembles an actual run of an experimental device: it starts by stabilising the cantilever, then the tip approaches the surface (e.g. by trying to match the frequency change setpoint) and makes a scan. To make the simulation as general as possible, it is split into elementary calculations associated with elementary cantilever movements called *stages*. For instance, a single approach to a preset frequency shift is a single stage; a scan along a certain direction keeping the frequency shift and the velocity is also an elementary stage. A more complex tip movement can always be *programed* as a set of elementary stages, e.g. a 2D scan can be constructed as a set of forward and backward line scans along two perpendicular directions.

When the SciFi and the vAFM work together, the full geometry optimisation in the junction and hence the calculation of the tip force is performed only when the tip is sufficiently close to the surface. At larger distances, a supplied analytical expression for the tip force is used. This speeds up the calculation enormously. In addition, the following “learning” feature is also implemented: if the lateral position of the tip does not change appreciably, the calculated force field is recorded, and interpolation of the force is made for a tip vertical position that lies between two known (already calculated) positions.

The whole code has been designed to run efficiently on both serial and distributed memory multi-processor machines with Fortran 77 or 90 compilers. In parallel the program uses the Message Passing Interface (MPI) set of communication routines and is paralised according to the distributed memory paradigm with atomic pair decomposition. At present, the code only works in parallel when vAFM option is switched off. This will be rectified shortly.

Part I

Theoretical Models

1 Semi-classical part (SciFi)

1.1 Introduction

Usually, the theoretical interpretation of AFM images is based on simple models in which the force imposed on the tip originates from the macroscopic long-range Van-der-Waals interaction and a microscopic interaction described using pair potentials between atoms simulating the tip apex and atoms of the sample. In this model only the direct electrostatic interaction between atoms of the tip apex and the sample is accounted for. However, the theoretical model used here takes account of the polarization of the conducting electrodes (i.e. of the tip and the substrate) by the charged atoms of the sample. This is important for any tip-surface (or just surface) setup containing conducting materials e.g. interaction of a conducting tip with an insulating surface or studying the properties of an insulating thin film on top of a metal substrate.

The potential on conducting electrodes is maintained by external sources (i.e. by the “battery”). From the point of view of classical electrostatics the polarization of the conductors by external charges is caused by the additional potential on the conductors due to the charges. This extra potential is compensated by a charge flow from one electrode to another to keep the potential on the conductors fixed. This work is done by the battery. As a result, there will be some distribution of the net charge on the surfaces of conductors induced by the point charges situated in the free space between them. The net charge on each conducting electrode would interact with the total charge on other conductors and with the point charges. Hereafter this interaction is called the *image interaction*. A detailed discussion of the derivation and application of the image interaction to AFM systems can be found in refs. [1] and [2].

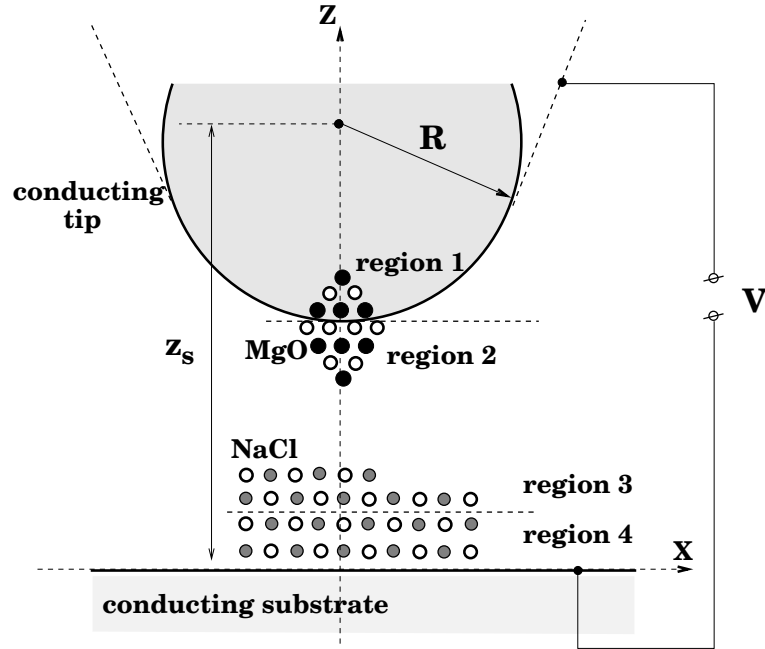


Figure 1: Example schematic picture of the microscopic model used to simulate the interaction between the tip and the sample.

The program itself is designed to calculate the interactions in systems similar to that shown in fig. 1, with an atomistic nano-tip embedded in a conducting macroscopic sphere interacting with an atomistic cluster on top of a metal substrate. This represents a very common experimental setup in modern AFM experiments using conducting tips to image thin insulating films on top of metal substrates. However, the code is very flexible in the types of setup it can study. The properties of insulating tips can be easily investigated by removing the conducting sphere from the system, and in fact the properties of thin films alone can be studied by removing the tip completely from

the system. Some example input files at the end of this manual show the variety of different setups that can be studied.

1.2 Electrostatic energy of a system of metals and charges

Consider a set of *finite* metallic conductors of arbitrary shape and an arbitrary distribution of point charges $\{q_i\}$ at the points $\{\mathbf{r}_i\}$ anywhere in the free space outside the conductors. We assume that the conductors which will hereafter be designated by indices m, m' are kept at some *fixed* potentials ϕ_m . These potentials are provided by the batteries.

It is known from the standard textbooks (see e.g. Refs. [3], [4], [5]) how to calculate the energy accumulated in the electrostatic field $\mathbf{E}$ created by point charges and metals. Using the total energy of the field, $U = \frac{1}{8\pi} \int_V \mathbf{E}^2 d\mathbf{r}$ (the integral is taken over the volume V outside the metals since inside those the field $\mathbf{E} = 0$) and applying the Poisson equation for the field, one gets:

$$U = \frac{1}{2} \sum_i q_i \phi(\mathbf{r}_i) + \frac{1}{2} \sum_m Q_m \phi_m \quad (1)$$

where the first sum is taken over all point charges whereas the second one over all conductors. $Q_m = -\frac{1}{4\pi} \int \int_{S_m} \frac{\partial \phi}{\partial n} ds$ is the charge on the metal m , where the integral is performed over the surface S_m of the metal. Note that the surface integral over a remote surface at infinity (which is surrounding all the metals and the point charges) vanishes due to rapid decrease of the potential to zero there [4]. Therefore, this derivation is not valid for infinite metals for which this assumption is not correct (e.g. a charged infinite metal conductor). We also note in passing that according to classical theory, the charge is distributed at the surface, i.e. it does not penetrate into the bulk of the metal; quantum theory [6], [7], [8] gives a certain distribution of the surface charge in the direction to the bulk.

The result of Eq. (1) has a very simple physical meaning: every charge q at the point $\mathbf{r}_q$ (either a point charge outside the metals or the distributed charge on a metal surface) gets energy $\frac{1}{2} q \phi(\mathbf{r}_q)$ where the factor $\frac{1}{2}$ is needed to avoid double counting. Note that the potential $\phi(\mathbf{r})$ is produced both by the metals (i.e. by the distributed charges on their surfaces) and by the point charges. Note also that both the potential $\phi(\mathbf{r}_i)$ on point charges and the charges Q_m on the metals are unknown and should be calculated by solving the Poisson equation. The effect of the metals comes into play via the boundary conditions and the charges Q_m .

In order to calculate the electrostatic force imposed on any of the conductors, we use the method essentially similar to the one of Ref. [4] where an arbitrary distribution of metals is considered without point charges. The force is obtained by differentiating the energy with respect to the corresponding position of the metal of interest. It is shown in Ref. [4] that the same expression for the force is obtained in the cases of fixed potentials or fixed charges on the metals. This, however, is not the case if the point charges are present. Indeed, let us move some metal by the vector $\delta \mathbf{r}$. The work $\delta A = -\mathbf{F} \delta \mathbf{r}$ is done by the external force against the force $\mathbf{F}$ imposed on the conductor. When the conductor is moved to the new position, the potential $\phi(\mathbf{r})$ in the system will change by $\delta \phi(\mathbf{r})$. The potential on any metal m will no longer be equal to the fixed value ϕ_m so that there should be some charge flow between the connected conductors to maintain the potential on them. Therefore, some work δA will be spent in changing the potential energy of the field (given by Eq. (1)) by the amount

$$\delta U = \frac{1}{2} \sum_i q_i \delta \phi(\mathbf{r}_i) + \frac{1}{2} \sum_m \phi_m \delta Q_m$$

and some work δA_b is done in transferring charge *between* the conductors. The latter work δA_b is done by the batteries (as the charge flows via the battery from one conductor to another) and so should be taken with the minus sign: $\delta A = -\delta A_b + \delta U$. Alternatively, one could think of the batteries being incorporated into the system; in that case it would mean that the work done by the batteries would reduce the total potential energy of the whole system.

Let δQ_m be the change of the charge on the conductor m in the discussing process. Then the work $\delta A_b = \sum_m \phi_m \delta Q_m$ (cf. Eq. (2.5) in Ref. [4]). Indeed, since $\sum_m \delta Q_m = 0$, this is the work needed to distribute the zero charge initially stored at infinity (where the potential is zero) between different metals by transferring the amounts δQ_m to the each metal m . Using the above given expressions, we obtain:

$$\delta A \equiv -\mathbf{F} \delta \mathbf{r} = -\sum_m \phi_m \delta Q_m + \delta U$$

so that the final expression for the force imposed on the displaced metal becomes

$$\mathbf{F} = -\frac{\partial U^{eff}}{\partial \mathbf{r}} \quad (2)$$

with the *effective energy* (or the total potential energy of the whole system which includes the batteries as well) defined as:

$$U^{eff} = \frac{1}{2} \sum_i q_i \phi(\mathbf{r}_i) - \frac{1}{2} \sum_m Q_m \phi_m \quad (3)$$

1.3 Tip-surface Interaction

In order to calculate the force imposed on the tip we used the following expression for the total energy of the system:

$$U = \frac{1}{2} \sum_{ij} v_{ij} + U_{vdW} + U_{el} \quad (4)$$

where v_{ij} is the interatomic potential between atoms i, j . This interaction energy is a function of both the tip position z_s and the coordinates $\mathbf{r}_i$ of all atoms (and shells) involved. The energy U_{vdW} represents the van der Waals interaction between the macroscopic tip and substrate and depends only on the tip height z_s with respect to the substrate (see Fig. 1). Note that in the present model the macroscopic van der Waals interaction does not effect atomic positions.

The last term in Eq. (4) represents the total electrostatic energy of the whole system. We assume that the Coulomb and covalent interactions between all atoms is included in the first term in Eq. (4) and hence should be excluded from U_{el} . Therefore, the energy U_{el} includes the interaction of the atoms in the system with the macroscopic tip and substrate. As has been demonstrated, the correct electrostatic energy should incorporate the work done by the battery to maintain the constant potential on the electrodes. It can be written in the following form:

$$U_{el} = -\frac{1}{2} Q^{(0)} V + \sum_i q_i \phi^{(0)}(\mathbf{r}_i) + \frac{1}{2} \sum_{i,j} q_i q_j \phi_{ind}(\mathbf{r}_i, \mathbf{r}_j) \quad (5)$$

Here V is the potential difference applied to the metal electrodes. Without loss of generality, one can choose the potential ϕ on the metal plane to be zero, so that the potential on the macroscopic part of the tip will be $\phi = V$. We also note that the substrate is considered in the limit of a sphere of a very big radius $R' \gg R$ since the metal electrodes formally cannot be infinite. The charge on the tip *without* charges outside the metals (i.e. when there are only bare electrodes and the polarization effects can be neglected) is $Q^{(0)}$ and the electrostatic potential of the bare electrodes anywhere outside the metals is $\phi^{(0)}(\mathbf{r})$. The charge $Q^{(0)}$ and the potential $\phi^{(0)}(\mathbf{r})$ depend only on the geometry of the capacitor formed by the two electrodes and on the bias V . The charge $Q^{(0)}$ can be calculated from the potential $\phi^{(0)}(\mathbf{r})$ as follows [4]: $Q^{(0)} = -\frac{1}{4\pi} \int \int \frac{\partial \phi^{(0)}}{\partial n} ds$ where the integration is performed over the entire surface of the macroscopic part of the tip with the integrand being the normal derivative of the potential $\phi^{(0)}(\mathbf{r})$; the normal $\mathbf{n}$ is directed outside the metal. Summation in the second term of Eq. (5) is performed over the atoms and shells of the sample and those of the tip apex which are represented by point charges q_i at positions $\mathbf{r}_i$.

Finally, $\phi_{ind}(\mathbf{r}, \mathbf{r}')$ in Eq. (5) is the potential at $\mathbf{r}$ due to image charges *induced* on all the metals by a unit point charge at $\mathbf{r}'$. This function is directly related to the Green function $G(\mathbf{r}, \mathbf{r}')$ of the Laplace equation, $\phi_{ind}(\mathbf{r}, \mathbf{r}') = G(\mathbf{r}, \mathbf{r}') - \frac{1}{|\mathbf{r} - \mathbf{r}'|}$, and is symmetric, i.e. $\phi_{ind}(\mathbf{r}, \mathbf{r}') = \phi_{ind}(\mathbf{r}', \mathbf{r})$, due to the symmetry of the Green function itself [5]. The total potential at $\mathbf{r}$ due to a net charge induced on all conductors present in the system by all the point charges $\{q_i\}$,

$$\phi_{ind}(\mathbf{r}) = \sum_i q_i \phi_{ind}(\mathbf{r}, \mathbf{r}_i) \quad (6)$$

is the *image potential*. Note that the last double summation in Eq. (5) includes the $i = j$ term as well. This term corresponds to the interaction of the charge q_i with its own polarization (similar to the polaronic effect in solid-state physics).

The function $\phi_{ind}(\mathbf{r}, \mathbf{r}')$ and, therefore, the image potential, $\phi_{ind}(\mathbf{r})$, together with the potential $\phi^{(0)}(\mathbf{r})$ of the bare electrodes could be calculated if we knew the exact Green function of the electrostatic problem

$$\Delta_{\mathbf{r}'} G(\mathbf{r}, \mathbf{r}') = -4\pi \delta(\mathbf{r} - \mathbf{r}') \quad (7)$$

with the corresponding boundary conditions ($G(\mathbf{r}, \mathbf{r}') = 0$ when $\mathbf{r}$ or $\mathbf{r}'$ belong to either the substrate or the tip surface) [5]. Therefore, given the applied bias V , the geometric characteristics of the capacitor and the positions $\{\mathbf{r}_j\}$ of the point charges $\{q_i\}$ between the tip and sample, one can calculate the electrostatic energy U_{el} . The problem is that the Green function for real tip-sample shapes and arrangements is difficult to calculate. In this study we use the planar-spherical geometry of the junction as depicted in Fig. 1.

1.4 Solution of the electrostatic problem of point charges inside the sphere-plane capacitor

First, let us consider the calculation of the potential $\phi^{(0)}(\mathbf{r})$ of the bare electrodes, i.e. the capacitor problem. We note that the potential $\phi^{(0)}(\mathbf{r})$ satisfies the same boundary conditions as the original problem, i.e. $\phi^{(0)} = 0$ and $\phi^{(0)} = V$ at the lower and upper electrodes, respectively. The solution for the plane-spherical capacitor is well known [9] and can be given using the method of image charges. Since we will need this solution for calculating forces later on, we have to give it here in detail. It is convenient to choose the coordinate system as shown in Fig. 1. Then it is easy to check that the following two *infinite* sequences of image charges give the potential at the sphere and the metal plane as V and zero respectively. The first sequence is given by the image charges $\varsigma_1 = RV$ and then $\varsigma_{k+1} = \frac{\varsigma_k}{D_k}$ for $\forall k = 1, 2, \dots$, where the dimensionless constants D_k are defined by the recurrence relation $D_{k+1} = 2\lambda - \frac{1}{D_k}$ with $D_1 = 2\lambda$ and $\lambda = \frac{z_s}{R} > 1$, z_s being the distance between the sphere centre and the plane (Fig. 1). The point charges $\{\varsigma_k\}$ are all inside the sphere along the normal line passing through the sphere centre. Their z -coordinates are as follows: $z_1 = z_s$ and $z_{k+1} = R\left(\lambda - \frac{1}{D_k}\right) = R(D_{k+1} - \lambda)$ for $\forall k = 1, 2, \dots$. The second sequence of charges $\{\varsigma'_k\}$ is formed by the images of the first sequence with respect to the metal plane, i.e. $\varsigma'_k = -\varsigma_k$ and $z'_k = -z_k$. An interesting point about the image charges $\{\varsigma_k\}$ is that they converge very quickly at the point $z_\infty = R\sqrt{\lambda^2 - 1}$ (i.e. $z_k \rightarrow z_\infty$ with $k \rightarrow \infty$) and that $z_{k+1} < z_k$ for $\forall k$. This is because the numbers D_k converge rapidly to the limiting value $D_\infty = \lambda + \sqrt{\lambda^2 - 1}$ which follows from the original recurrent relation above, $D_\infty = 2\lambda - \frac{1}{D_\infty}$. Therefore, while calculating the potential $\phi^{(0)}(\mathbf{r})$, one can consider the charges $\{\varsigma_k\}$ and $\{\varsigma'_k\}$ explicitly only up to some $k = k_0 - 1$ and then sum up the rest of the charges to infinity analytically to obtain the effective charge $\varsigma_\infty = \sum_{k=k_0}^{\infty} \varsigma_k = \sum_{n=0}^{\infty} \frac{\varsigma_{k_0}}{D_\infty^n} = \frac{\varsigma_{k_0} D_\infty}{D_\infty - 1}$ to be placed at z_∞ . This can be used *instead* of the rest of the series:

$$\phi^{(0)}(\mathbf{r}) = \sum_{k=1}^{k_0} \bar{\varsigma}_k \left(\frac{1}{|\mathbf{r} - \mathbf{r}_{\varsigma_k}|} - \frac{1}{|\mathbf{r} - \hat{\sigma}\mathbf{r}_{\varsigma_k}|} \right) \quad (8)$$

where $\bar{\varsigma}_k = \varsigma_k$ for $k < k_0$ and $\bar{\varsigma}_{k_0} = \varsigma_\infty$; then, $\mathbf{r}_{\varsigma_k} = (0, 0, z_k)$ is the position vector of the charge ς_k and $\mathbf{r}_{\varsigma'_k} = \hat{\sigma}\mathbf{r}_{\varsigma_k} = (0, 0, -z_k)$ is the position of the charge ς'_k ($\hat{\sigma}$ means reflection with respect to the substrate surface $z = 0$). To find the charge $Q^{(0)}$ which is also needed for the calculation of the electrostatic energy, Eq. (5), one should calculate the normal derivative of the potential $\phi^{(0)}(\mathbf{r})$ on the sphere and then take the corresponding surface integral (see above). However, it is useful to recall that the total charge induced on the metal sphere due to an external charge is equal exactly to the image charge inside the sphere [4], [5]. Therefore, one immediately obtains:

$$Q^{(0)} = \sum_{k=1}^{k_0} \bar{\varsigma}_k \quad (9)$$

Note that the potential $\phi^{(0)}(\mathbf{r})$ and the charge $Q^{(0)}$ depend on the position z_s of the sphere *indirectly* via the charges ς_k and their positions $\mathbf{r}_{\varsigma_k}$ according to the recurrent expressions above. Therefore, one has to be careful when calculating the contribution to the force imposed on the tip due to bias V (i.e. when differentiating $\phi^{(0)}(\mathbf{r})$ and $Q^{(0)}$ in Eq. (5)).

Now we turn to the calculation of the function $\phi_{ind}(\mathbf{r}, \mathbf{r}')$ in Eq. (5). This function corresponds to the image potential at a point $\mathbf{r}$ due to a unit charge at $\mathbf{r}'$. This potential is to be defined in such a way that together with the direct potential of the unit point charge, it should be zero on both electrodes (the boundary condition for the Green function). Thus, let us consider a unit charge $q = 1$ at $\mathbf{r}_q$ somewhere outside the metal electrodes as shown in Fig. 2. We first create the direct image $-q$ of this charge with respect to the plane at the point $\mathbf{r}'_q = \hat{\sigma}\mathbf{r}_q$ to maintain zero potential at the plane. Then, we create images of the two charges, $q = 1$ and $-q = -1$, with respect to the sphere to get two image charges $\xi_1 = -\frac{R}{|\mathbf{r}_q - \mathbf{R}_s|}$ and $\zeta_1 = \frac{R}{|\hat{\sigma}\mathbf{r}_q - \mathbf{R}_s|}$, as shown in Fig. 2 where $\mathbf{R}_s = (0, 0, z_s)$. These image charges are both inside the sphere by construction and their positions can be written down using a vector function $\mathbf{f}(\mathbf{r}) = \mathbf{R}_s + R^2 \frac{\mathbf{r} - \mathbf{R}_s}{|\mathbf{r} - \mathbf{R}_s|^2}$ as follows: $\mathbf{r}_{\xi_1} = \mathbf{f}(\mathbf{r}_q)$ and $\mathbf{r}_{\zeta_1} = \mathbf{f}(\hat{\sigma}\mathbf{r}_q)$. Now the potential at the surface will be zero. At the next step we construct the images $\xi'_1 = -\xi_1$ and $\zeta'_1 = -\zeta_1$ of the charges ξ_1 and ζ_1 in the plane at points $\hat{\sigma}\mathbf{r}_{\xi_1}$ and $\hat{\sigma}\mathbf{r}_{\zeta_1}$ respectively, to get the potential at the plane also zero. This process is continued and in this way two infinite sequences of image charges are constructed, which are given by the following recurrence relations:

$$\begin{cases} \xi_{k+1} = \xi_k \frac{R}{|\hat{\sigma}\mathbf{r}_{\xi_k} - \mathbf{R}_s|} \\ \mathbf{r}_{\zeta_{k+1}} = \mathbf{f}(\hat{\sigma}\mathbf{r}_{\xi_k}) = \mathbf{R}_s + R^2 \frac{\hat{\sigma}\mathbf{r}_{\xi_k} - \mathbf{R}_s}{|\hat{\sigma}\mathbf{r}_{\xi_k} - \mathbf{R}_s|^2} \end{cases} \quad (10)$$

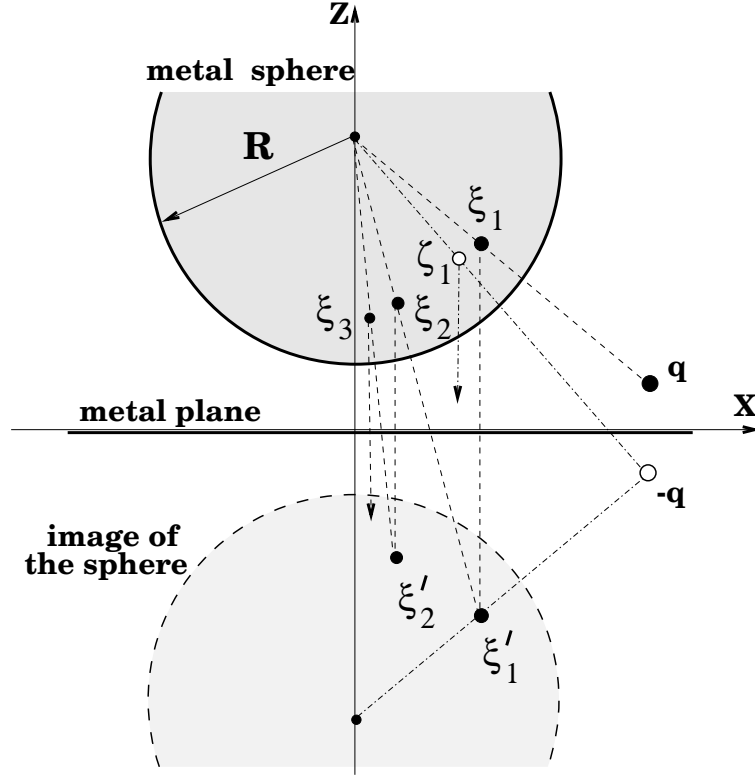


Figure 2: Construction of image charges in the sphere-plane capacitor system due to one charge q outside the metals.

where $k = 1, 2, \dots$ and similarly for the ζ -sequence. Note, however, that the two sequences start from different initial charges. Namely, the ξ -sequence starts from the original charge q while the ζ -sequence from its image in the plane $-q$. The two sequences $\{\xi_k\}$ and $\{\zeta_k\}$ are to be accompanied by the other two sequences $\{\xi'_k\}$ and $\{\zeta'_k\}$ which are the images of the former charges with respect to the plane. The four sequences of the image charges and the charges q and $-q$ provide the correct solution for the problem formulated above since they produce the potential which is the solution of the corresponding Poisson equation and, at the same time, is zero both on the metal sphere and the metal plane.

It is useful to study the convergence properties of the sequences of image charges. To simplify the notations, let us assume that the original charge is in the xz -plane. Then it follows from Eq. (10) that $x_{\xi_{k+1}} < x_{\xi_k}$. It is also seen that $x_{\xi_k} \rightarrow 0$ and $z_{\xi_k} \rightarrow z_\infty = R\sqrt{\lambda^2 - 1}$ (see above) as $k \rightarrow \infty$ and the same for the ζ -sequence. This means that the image charges inside the sphere move towards the vertical line passing through the centre of the sphere and finally converge at the same point z_∞ . This is the same behaviour we observed for charges in the capacitor problem at the beginning of this subsection (see Fig. 2). In fact, the calculation clearly shows a very fast convergence so that we can again sum up the series of charges from some $k = k_0$. Thus, the image potential at a point $\mathbf{r}$ due to the unit charge at $\mathbf{r}_q$ is:

$$\phi_{ind}(\mathbf{r}, \mathbf{r}_q) = -\frac{1}{|\mathbf{r} - \widehat{\mathbf{r}}_q|} + \sum_{k=1}^{k_0} \left[\bar{\xi}_k \left(\frac{1}{|\mathbf{r} - \mathbf{r}_{\xi_k}|} - \frac{1}{|\mathbf{r} - \widehat{\mathbf{r}}_{\xi_k}|} \right) + \bar{\zeta}_k \left(\frac{1}{|\mathbf{r} - \mathbf{r}_{\zeta_k}|} - \frac{1}{|\mathbf{r} - \widehat{\mathbf{r}}_{\zeta_k}|} \right) \right] \quad (11)$$

where $\bar{\xi}_k = \xi_k$ for $k < k_0$ and $\bar{\xi}_{k_0} = \xi_\infty = \xi_{k_0} \frac{D_\infty}{D_\infty - 1}$, and similarly for the ζ -sequence. Here D_∞ is the geometrical characteristic of the capacitor introduced at the beginning of this subsection.

As it has been already mentioned in Section 2.1, the function $\phi_{ind}(\mathbf{r}, \mathbf{r}_q)$ must be symmetric with respect to the permutation of its two variables. It is not at all obvious that this is the case since the meaning of its two arguments in Eq. (11) is rather different. Nevertheless, we show in ref. [2] that the function $\phi_{ind}(\mathbf{r}, \mathbf{r}_q)$ is symmetric.

1.5 The calculation of the total force acting on the tip

In order to calculate the total force acting on the tip, one has to differentiate the total energy, Eq. (4), with respect to the position of the sphere, $\mathbf{R}_s$. Since we are interested only in the force acting in the z -direction, it is sufficient to study the dependence of the energy on z_s . There will be three contributions to the force. The second contribution to the force comes from the van der Waals interaction [10]. Finally, the interatomic interactions lead to a force which is calculated by differentiating the shell-model energy (the first term in Eq. (4)). Therefore,

$$F_{tip} = -\frac{dU_{sh}}{dz_s} - \frac{dU_{vdW}}{dz_s} - \frac{dU_{el}}{dz_s} \quad (12)$$

where $U_{sh} = \frac{1}{2} \sum_{ij} v_{ij}$ is the shell model energy. We recall that the summation here is performed over all atoms in regions 1 to 4. Only positions of the atoms in region 1 depend directly on z_s since atoms in other regions are allowed to relax. However, their equilibrium positions, $\mathbf{r}_i^{(0)}$, determined by the minimisation of the energy of Eq. (4), will depend *indirectly* on z_s at equilibrium, $\mathbf{r}_i^{(0)} = \mathbf{r}_i^{(0)}(z_s)$. Then, we also recall that the electrostatic energy, U_{el} , depends only on positions of atoms, as well as on the tip position z_s .

Let us denote the positions of atoms by a vector $\mathbf{x} = (\mathbf{r}_1, \mathbf{r}_2, \dots)$. The total energy $U = U(\mathbf{x}, z_s)$, where the direct dependence on z_s comes from relaxed tip atoms of the shell model energy, U_{sh} , and from the electrostatic energy, U_{el} . In equilibrium the total energy is a minimum:

$$\left(\frac{\partial U}{\partial \mathbf{x}} \right)_{z_s} = 0 \quad (13)$$

where the derivatives are calculated at a given fixed tip position z_s . Let $\mathbf{x}_0 = (\mathbf{r}_1^{(0)}, \mathbf{r}_2^{(0)}, \dots)$ be the solution of Eq. (13). Then, since $\mathbf{x}_0 = \mathbf{x}_0(z_s)$, we have for the force:

$$F_{tip} = -\frac{dU(\mathbf{x}_0(z_s), z_s)}{dz_s} = -\left(\frac{\partial U}{\partial \mathbf{x}_0} \right)_{z_s} \frac{\partial \mathbf{x}_0}{\partial z_s} - \left(\frac{\partial U}{\partial z_s} \right)_{\mathbf{x}_0} = -\left(\frac{\partial U}{\partial z_s} \right)_{\mathbf{x}_0} \quad (14)$$

where we have used Eq. (13). This result can be simplified further. Indeed, the partial derivative of the shell-model energy, $-\frac{\partial U_{sh}}{\partial z_s}$, is equal to the sum of all z -forces acting on atoms in region 1 due to all shell-model interactions, since only these atoms are responsible for the dependence on z_s in the energy U_{sh} . Therefore, finally we have:

$$F_{tip} = \sum_{i \in 1} \left(F_{iz}^{(sh)} \right)_{\mathbf{x}_0} - \frac{dU_{vdW}}{dz_s} - \left(\frac{\partial U_{el}}{\partial z_s} \right)_{\mathbf{x}_0} \quad (15)$$

where the first summation runs only over fixed tip atoms. Thus, in order to calculate the force imposed on the tip at a given tip position z_s , one has to relax the positions of atoms using the *total energy* of the system, $U_{sh} + U_{el}$. Then calculate the shell-model force, $F_{iz}^{(sh)}$, acting on every fixed tip atom in the z -direction as well as the electrostatic contribution to the force given by the last term in Eq. (15). The van der Waals force between macroscopic tip and sample does not depend on the geometry of atoms and can be calculated just once for every given z_s .

1.5.1 Definition of the Instantaneous Force on the Tip (for MD)

In the case where the system is dynamic and atoms are given kinetic energy, it becomes obvious that the definition of the tip force given in the previous section no longer holds, since the forces on free atoms are no longer zero: $\frac{\partial U}{\partial \mathbf{r}} \neq 0$. It is also clear that because of this when the entire tip is far from the surface, region 1 will experience large fluctuations in force of Eq. (14) due to oscillating other tip atoms belonging to the adjacent region 2. Moreover, these fluctuations will not decay with the distance from the surface which is in clear contradiction with what one would expect from the tip force which is physically due to the interaction with the surface. Even for a completely isolated tip the force fluctuations are large although, since region 2 atoms vibrate around equilibrium positions, the average force fluctuates around zero value. Hence, we conclude that the final form of Eq. (14) can only be used for calculating the force in mechanical equilibrium or when calculating the *average* force during MD simulations. If we are interested in the actual *fluctuations* of the tip force for any given tip position Q , Eq. (14) cannot be applied.

What is needed in order to provide an alternative expression for the tip force which would work for both static and dynamic situations, is the definition of an *instantaneous* force for arbitrary positions of atoms in the junction. At first, one might be tempted to define the tip force as the partial derivative, $-\frac{\partial U}{\partial Q}$, of the total system energy U with respect to tip height Q at *arbitrary* positions of atoms, $\mathbf{r}$, which are allowed to relax. However, this definition,

as can easily be seen, leads to the same expression for the force, Eq. (14), because only atoms in region 1 depend explicitly on Q , and thus cannot be accepted.

In order to obtain an appropriate definition of the tip force, we invoke a general non-equilibrium microscopic consideration in which the total Hamiltonian of the “tip - surface” system was shown [11] to be split as follows:

$$H = H_s + \Phi + H_T \quad (16)$$

Here H_s is the sum of the independent *microscopic* Hamiltonians of the isolated surface and the isolated tip. Either of those contains kinetic and potential energies of the constituent atoms. Note that atomic positions in the tip part of H_s are defined *relatively* to the fixed part of the tip (e.g. using internal coordinates). Note also that there is *no* interaction between atoms in the surface and in the tip included in H_s . This interaction is completely included in the second term, Φ , in Eq. (16). Since absolute positions of the tip atoms are to be used while specifying the tip-surface interaction, Φ depends both on the atomic positions and the tip height, Q . As the distance between the tip and surface increases, the tip-surface interaction Φ tends to zero. Finally, H_T is the *macroscopic* Hamiltonian of the tip which contains kinetic and elastic energy of the cantilever as well as the excitation term. It depends exclusively on Q and also contributes to the conservative part of the tip force. Since we are not interested in the conservative contribution (it does not contribute to the force fluctuations), it will be ignored in this presentation. Note that the interaction term, Φ can always be defined for arbitrary complex many body interactions between the tip and surface, and thus our treatment is not limited to systems consisting only of pairwise interactions.

It then follows from the microscopic non-equilibrium theory that the instantaneous microscopic force acting on the tip (which completely includes the *stochastic* contribution due to atomic vibrations [12]) should be defined as follows:

$$F = -\frac{\partial \Phi}{\partial Q} = f_{13} + f_{23} \quad (17)$$

where f_{ij} is the sum of the vertical forces acting on all the atoms in region i due to all of the atoms in region j [13]. The important difference with the static definition of Eq. (14) is that the force is defined as the partial derivative of the *interaction* energy Φ rather than the *total* energy U . We see that the microscopic force on the tip is given as the sum of the *partial forces* on *all* atoms of the tip due to all of the atoms in the surface.

The above expression does not require that the system is in equilibrium, and, due to partitioning of the total Hamiltonian discussed above, Eq. (16), it naturally tends to zero when the tip and the surface are separated. It is also equivalent to the static definition (14) if the system is in equilibrium. Indeed, since $f_{21} = -f_{12}$, $f_{22} = 0$ and $f_{11} = 0$ at any given time t , we can also write F as

$$F = (f_{23} + f_{22} + f_{21}) + (f_{13} + f_{12} + f_{11}) \quad (18)$$

The first term (all three terms in the first brackets) is equal to the sum of the total forces acting on atoms in region 2 due to all other atoms in the system; it is equal to zero in mechanical equilibrium. Thus, in this case F reduces to the second term (the second brackets) which is equal to the sum of forces acting on the frozen atoms of the tip comprising region 3 which is indeed identical to that given by Eq. (14).

In practical calculations the expression for the force (17) is not very convenient since it requires calculation of partial forces due to different regions to be recorded during each simulation step. The equivalent expression (18), which will be referred to as the *dynamic* definition of the tip force (as opposed to the *static* definition of Eq. (14)) is much more convenient since it contains the sum of total forces acting on *all* atoms of the tip. This expression is used in the code when calculating the continuous force on the tip during a simulation.

1.6 Interatomic Potentials

All atoms of the system in SciFi are divided into three non-overlapping groups, some of the groups may be completely missing:

- **pair-wise atoms**, i.e. atoms interacting between themselves via pair-wise interaction potentials, see Section 1.6.1;
- **many-body atoms**, which interact between themselves via many-body force fields (e.g. organic molecules), see Section 1.6.2;
- **Tersoff atoms** interacting with each other via a many-body force field developed by Tersoff [14] (from version 4.0).

Interaction between atoms belonging to different groups can only be performed via a pairwise interaction. Also, no additional pairwise interaction can be present between Tersoff atoms or between many-body atoms. Electronic polarisation may be accounted for via the shell model (see below). However, no shell are allowed on the Tersoff atoms. All other atoms may have shells. For instance, manybody atoms may have them in which case interaction of these with pair-wise atoms will include electronic polarisation effects.

The potential energy surface of the system is modelled by the following force field:

$$E = E_{pw} + E_{mb} + E_{Tf} + E_{image} \quad (19)$$

where E_{pw} is the contribution from all present pair-wise interactions in the system including the electronic polarization; E_{mb} is the interaction energy between many-body atoms; E_{Tf} is the interaction between Tersoff atoms; finally, E_{image} is the contribution to the total energy due to applied bias and the polarization of electrodes as described in the Sections above.

Normally, calculations are performed on a system which is finite in all three directions (a finite cluster). However, if Tersoff atoms are present in the system, then it is possible to run 2D and 3D periodic calculations as well (i.e. in **Periodic Boundary Conditions**, PBC), i.e. atoms within the central cell are surrounded by their images. Only Tersoff atoms are mirrored, so that if there are any many-body (e.g. a molecule) or pair-wise atoms also present, they do not know about their images, there is no interaction neither between their images nor between them and the images of the Tersoff atoms.

The code also allows the forces on atoms calculated analytically to be checked by a numerical interpolation of the energy (see Section 8.2.4).

1.6.1 Pairwise interaction

The pair-wise interaction is modelled as follows:where

$$E_{pw} = \sum_{ij} \frac{q_i q_j}{r_{ij}} + \sum_{ij} C_{ij} r_{ij}^{-n_{ij}} + \sum_{ij} A_{ij} e^{-\frac{r_{ij}}{\rho_{ij}}} \quad (20)$$

where the indices i, j run over all atoms (and their shells) which contribute to this interaction.

The program implements the Shell Model to calculate pairwise interactions, first introduced in [15]. In this scheme, atoms are represented by positively charged cores connected to negatively charged shells by harmonic springs, which represent nuclear and electron charge respectively. The equilibrium distance between the core and shell is a representation of the electronic polarization of that atom. Cores and shells of the same atom only interact via the harmonic spring, however each atomic core and shell interacts via colombic forces with all other cores and shells in the system.

Short range repulsive and Van de Waals forces are described by pairwise Buckingham potentials (last two terms in the above expression), which act only between shells.

If shell are not present, all interactions act between cores which have the actual charges $\{q_i\}$ on atoms.

1.6.2 Many-body interactions

Many body interactions are used to model covalently bonded crystals and organic molecules in the system. The energy expression implemented in the code:

$$E_{mb} = \sum_b^{N_b} K_b (b - b_0)^2 + \sum_\theta^{N_\theta} K_\theta (\theta - \theta_0)^2 + \sum_\phi^{N_\phi} K_\phi [1 + \cos(n\phi - \phi_0)] + \sum_\chi^{N_\chi} K_\chi [1 + \cos(n\chi - \chi_0)] \quad (21)$$

The terms in the above expression apply to the many body interactions in the system and summations only apply to the specified interactions, with the parameters supplied by the user. These interactions are most conveniently specified via the internal coordinates of the system, not via the Cartesian coordinates of atoms as is the case for other interactions. These are: bond lengths b , angles between bonds θ , torsions about bonds ϕ and out-of-plane angles χ . These are calculated using the expressions given in Figs. 3, 4 and 5. The internal coordinates are defined between the cores, and not shells, of atoms.

The length of a bond between two atoms in a molecule is straightforward to visualise, and is simply calculated as the magnitude of the vector ($\mathbf{r}$) between the two atoms (i and j) (Figure 3).

The angle between two adjacent bonds in a molecule, or the angle formed by the three nuclei i , j and k is calculated using the dot product of the two vectors from j to i (r_{ji}) and j to k (r_{jk}) as shown in figure 4.

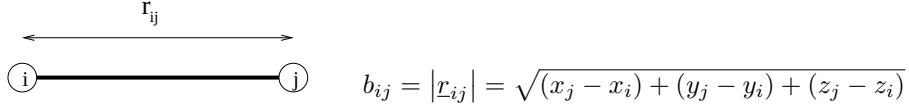


Figure 3: Bond Length

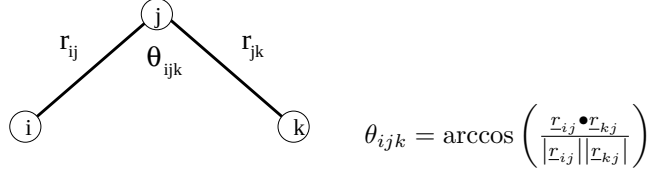


Figure 4: Bond Angle

A torsion angle about a bond $j - k$ is defined for any four sequentially bonded atoms in a molecule, i, j, k and l and made by the angle between the planes formed by the two angles θ_{ijk} and θ_{jkl} (Figure 3.4). The torsion angle is then calculated by taking the dot product of the normal unit vectors of these two planes, each of which is calculated from the vector cross products of $\underline{r}_{ji}$ with $\underline{r}_{jk}$ and of $\underline{r}_{jk}$ with $\underline{r}_{kl}$ as shown in figure 5.

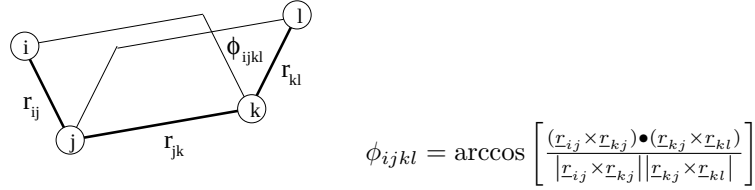


Figure 5: Torsion Angle

An out-of-plane angle needs to be defined when an atom in a molecule is bonded to just three other atoms (central atom i bonded to atoms j, k and l), and the out-of-plane angle (χ) is defined as the average of the three improper torsions formed by the interaction.

The improper torsion definition uses the torsion angle expression above to calculate the torsion angle formed by the atoms $i - k - l - j$, which is the angle between the plane formed by the three atoms i, k, l and the plane formed the atoms k, l and the central atom i . It is so called improper as the torsion is about two atoms that are not bonded together.

The analytical derivatives of these expressions were used in the previous versions of the code to calculate the gradients of the energy analytically. However, after the PBC was introduced (from version 4.0), forces are calculated numerically (this only belongs to the Tersoff interaction).

1.6.3 Tersoff interactions

Interaction energy between Tersoff atoms is given according to the following equations:

$$E_{Tf} = \frac{1}{2} \sum_i \sum_{j \in nn(i)} F_C(r_{ij}) [F_R(r_{ij}) - b_{ij} F_A(r_{ij})] \quad (22)$$

where

$$F_R(r_{ij}) = A_{ij} e^{-\lambda_{ij}^{(1)} r_{ij}}, \quad F_A(r_{ij}) = B_{ij} e^{-\lambda_{ij}^{(2)} r_{ij}} \quad (23)$$

$$F_C(r_{ij}) = \begin{cases} 0, & \text{if } r_{ij} > R_{ij} + D_{ij} \\ 1, & \text{if } r_{ij} < R_{ij} - D_{ij} \\ \frac{1}{2} \left[1 - \sin \left(\frac{\pi(r_{ij} - R_{ij})}{2D_{ij}} \right) \right], & \text{otherwise} \end{cases} \quad (24)$$

$$b_{ij} = \chi_{ij} [1 + (\beta_i \xi_{ij})^{n_i}]^{-1/2n_i} \quad (25)$$

$$\xi_{ij} = \sum_{k \in nn(i), k \neq j} F_C(r_{ik}) g(\theta_{ijk}) e^{[\lambda_{ijk}^{(3)} (r_{ij} - r_{ik})]^3} \quad (26)$$

$$g(\theta_{ijk}) = 1 + \left(\frac{c_i}{d_i}\right)^2 - \frac{c_i^2}{d_i^2 + (h_i - \cos(\theta_{ijk}))^2} \quad (27)$$

$$\cos(\theta_{ijk}) = \frac{r_{ij}^2 + r_{ik}^2 - r_{jk}^2}{2r_{ij}r_{ik}}$$

The second summation in Eq. (22) (over j) is performed over all nearest neighbours (nn) of atom i . The summation over k in Eq. (26) is run over all nearest neighbours of i excluding atom j . Here $F_C(r)$ is a smoothing function. All other quantities are parameters. In spite of the apparently pair-wise form, Eq. (22), this is a many-body interaction in nature because of the way the coefficients b_{ij} are calculated in Eqs. (25) - (27).

The code contains default values of all parameters for Si, Ge and C. The user may change the parameters, however.

1.7 Constrained minimisation

The code can perform constrained minimisation searches¹. A general structure implemented actually allows for very sophisticated nonlinear constraints, but those in general require some additional coding which is described in some detail in the corresponding routines as documented in Table 1: We shall first explain the main ideas of very general

routine	file	comment	extra coding?
<code>constraints()</code>	<code>inval.f</code>	reading from input file; setting up	Yes
<code>check_constr()</code>	<code>inval.f</code>	checking all constraints for consistency	May be
<code>force_conv_3_1()</code>	<code>shell.f</code>	converts forces <code>fx,fy,fz</code> => 1 dim vector	Yes
<code>coord_constr()</code>	<code>shell.f</code>	deals with redundant coordinates	Yes
<code>conv_1_3()</code>	<code>shell.f</code>	converts coordinates to <code>x,y,z</code> from 1 dim vector	No
<code>conv_3_1()</code>	<code>shell.f</code>	converts coordinates <code>x,y,z</code> => 1 dim vector	No
<code>do_restart()</code>	<code>restart.f</code>	dumps an input RST file	Yes
-	<code>constr.inc</code>	memory storage/definitions	Yes

Table 1: Routines and the corresponding files responsible for the interface dealing with imposed constraints. The last column states if any extra coding is required when an additional constraint is to be added.

constraints; then linear constraints which are implemented in a rather general way in the code will be discussed specifically.

1.7.1 How constraints are dealt with in the code

Constraints are dealt with in the following general way: every constrained equation specifies a single atomic coordinate of some atom (say, coordinate x of atom 3, i.e. x_3) which is a function of other coordinates of the same and/or some other atoms, i.e. $x_3 = \xi(y_3, z_1, x_2)$. The coordinate x_3 is then made *redundant* and is removed from the optimisation space. The consequence of this is that, when calculating the “force” in the optimisation space associated with other coordinates which are involved in the given constraint, there will be an additional contribution due to the constraint. In the example above coordinates y_3 , z_1 and x_2 are involved, so that the energy

$$E = E(x_1, y_1, z_1, x_2, y_2, z_2, x_3(y_3, z_1, x_2), y_3, z_3) \equiv \tilde{E}(x_1, y_1, z_1, x_2, y_2, z_2, y_3, z_3)$$

is a function of all coordinates except for x_3 . Therefore, the “force” associated with this coordinate is not needed during optimisation. However, e.g. the “force” associated with y_3 becomes:

$$\tilde{f}_{3y} = -\frac{\partial \tilde{E}}{\partial y_3} = -\frac{\partial E}{\partial y_3} - \frac{\partial E}{\partial x_3} \frac{\partial x_3}{\partial y_3} = f_{3y} + f_{3x} \frac{\partial x_3}{\partial y_3} = f_{3y} + f_{3x} \frac{\partial \xi}{\partial y_3}$$

where f_{3y} and f_{3x} are the actual (physical) forces on atom 3 in the y and x directions, respectively. These are calculated routinely in the code irrespective of the constraints. The derivative $\frac{\partial \xi}{\partial y_3}$ needed for the calculation of the “force” $\tilde{f}_{3y}$ should be calculated analytically for every particular constraint from the known function ξ and then coded in in the appropriate places.

¹Formally, constraints can also be used during MD runs, but those should be done with care and generally we do not recommend this approach unless you know what you are doing.

There is presently only one important limitation on the atoms involved in constraints: a redundant coordinate must be contained in only one constraint; it is not allowed that it is a member of another constraint as well. However, other coordinates are allowed to participate in several constraints.

A note for programmers: coordinates allowed to relax have their relaxation flags (`iRx`, `iRy`, `iRz`) fixed to 1; coordinates not allowed to relax have the flags equal to 0; redundant coordinates, however, have their flags equal to 2. Gradients are calculated for all atoms which have their flags larger than zero.

1.7.2 Linear constraints

In the case of linear constraints atomic coordinates of atoms (either different, the same or both) may be related linearly to each other. Let us collect all those related atomic Cartesian coordinates into a set (r_i) (i is not an atom number, but rather a number in the set). Then, the linear relationship between members of the set can be expressed as follows:

$$\sum_i C_i r_i = 0 \quad (28)$$

where C_i are some constants defined by the nature of the specific constraint. Note that not all atoms and all x , y , z components of atomic coordinates may be present. In practice several equations like that given above can be supplied. Different constraints will be distinguished by an additional index k .

The simplest example is to allow atoms to move only along certain directions. Simply by specifying relaxation flags `iRx`, `iRy`, `iRz` in the input (opposite to atomic coordinates, see section 8.4.5) one can already allow atoms to move only along x , y or z axes or in xy , yz or xz planes; additional constraints are not required in all those cases. However, imagine you would like to allow, say, atom 5 to move only along the (111) direction, i.e. to have $x_5 = y_5 = z_5$. In this case you will have to specify the following 2 constraints:

$$x_5 - y_5 = 0 \quad \text{and} \quad z_5 - y_5 = 0$$

Obviously, there is only one independent coordinate in this case. If x_5 is considered as the redundant coordinate from the first constraint and z_5 - from the second (since the coordinate y_5 must not be chosen as a redundant coordinate as it appears in both conditions), the only “active” coordinate in this case will be y_5 . In more complicated cases different atoms may be involved as well.

The first coordinate in the constraints equations like Eq. (28) with nonzero coefficient $C_1^{(k)} \neq 0$, as supplied in the input (section 8.10), is treated automatically as the redundant variable $r_1^{(k)}$ for the constraint k . This allows to express the redundant variable via all others involved in the given constraint

$$r_1^{(k)} = - \sum_{i \neq 1} \frac{C_i^{(k)}}{C_1^{(k)}} r_i^{(k)} \quad (29)$$

and then calculate the correction $\Delta_j^{(k)}$ to the “force” $\tilde{f}_j$ on any of the nonredundant atoms j due to the constraint k as:

$$\Delta_j^{(k)} = \frac{\partial E}{\partial r_1^{(k)}} \frac{\partial r_1^{(k)}}{\partial r_j} = -f_1^{(k)} \frac{C_{i(j)}^{(k)}}{C_1^{(k)}}$$

where $i \equiv i(j)$ is the position of atom j in the list of atoms involved in the constraint k . The total “force” is obtained by adding corrections $\Delta_j^{(k)}$ due to all constraints to the physical force f_j .

1.7.3 Constant distance between two atoms

Another simple (but nonlinear) constraint which has been implemented is to keep a distance, d , between two atoms constant. If, say, atoms 1 and 2 are involved, then the x component of the first atom specified in the input (section 8.10) is considered as redundant. It is expressed via other coordinates as follows:

$$x_1 = x_2 \pm \sqrt{d^2 - (y_1 - y_2)^2 - (z_1 - z_2)^2}$$

where two choices of the sign are possible. The correction to the “force” associated with nonredundant coordinates y_1 , z_1 and x_2 , y_2 , z_2 is calculated correspondingly.

1.7.4 Constant angle between three atoms

If angle α between three atoms, say atoms 1, 2 and 3, is to be kept constant, then the redundant coordinate is chosen to be the x coordinate of the first atom in the input (section 8.10). It is expressed via other 8 coordinates as follows:

$$x_1 = x_2 + \frac{1}{x_{32}^2 - r_{32}^2 \cos^2 \alpha} \left\{ -x_{32}d \pm \sqrt{(x_{32}d)^2 - (x_{32}^2 - r_{32}^2 \cos^2 \alpha)(d^2 - \Delta_{12}^2 r_{32}^2 \cos^2 \alpha)} \right\}$$

where $d = y_{12}y_{32} + z_{12}z_{32}$, $\Delta_{12}^2 = y_{12}^2 + z_{12}^2$, $r_{32}^2 = x_{32}^2 + y_{32}^2 + z_{32}^2$ and $x_{32} = x_3 - x_2$, etc. The correction to “forces” are straightforward to calculate; they look very cumbersome and not yet implemented.

1.8 Molecular Dynamics Algorithms

If a Molecular Dynamics (MD) simulation is being performed, then the trajectories of atoms are integrated at each timestep using the standard Verlet Leapfrog algorithm [16] in the NVE ensemble, and a variation of it if a thermostat is used to simulate in the NVT ensemble. The different algorithms used are described in the following section.

1.8.1 NVE Verlet Leapfrog

The standard Verlet Leapfrog algorithm first generates new velocities at timestep $n + \frac{1}{2}$ from the velocities at timestep $n - \frac{1}{2}$ and the forces on all atoms as follows:

$$\underline{v}(t + \frac{1}{2}\Delta t) \leftarrow \underline{v}(t - \frac{1}{2}\Delta t) + \Delta t \frac{\underline{f}(t)}{m}$$

Then the positions of all atoms are advanced one timestep based on the new velocities:

$$\underline{r}(t + \Delta t) \leftarrow \underline{r}(t) + \Delta t \underline{v}(t - \frac{1}{2}\Delta t)$$

The temperature of the system at time t is then calculated as the average of the temperatures of the system at timestep $t - \frac{1}{2}\Delta t$ and $t + \frac{1}{2}\Delta t$.

The initial velocities for the system can optionally be provided by the user in the input file, however if they are not the program assigns randomly oriented velocities to each of the free atoms in the system based on the given system temperature, ensuring that the total translational and rotational angular momentum of the system is zero.

1.8.2 NVT with Gaussian Thermostat

If the system is to be simulated in the NVT ensemble, then the user has the choice of four different algorithms, the Gaussian thermostat [17], Berendsen thermostat, the Nose-Hoover thermostat [18] and stochastic boundary conditions [19]. The first of these, the Gaussian constraint thermostat, rescales the velocities of atoms at each timestep to achieve a constant system temperature, however this algorithm will destroy dynamic correlations very quickly. The basic algorithm is detailed below:

$$\underline{v}(t + \frac{1}{2}\Delta t) \leftarrow (2\eta - 1)\underline{v}(t - \frac{1}{2}\Delta t) + \eta\Delta t \frac{\underline{f}(t)}{m}$$

$$\underline{r}(t + \Delta t) \leftarrow \underline{r}(t) + \Delta t \underline{v}(t - \frac{1}{2}\Delta t)$$

$$\eta = \sqrt{\frac{T_{ext}}{T}}$$

Where, T is the temperature at timestep t (that is calculated self consistently with three iterations) and T_{ext} is the given system temperature or bath temperature.

1.8.3 NVT with Berendsen Thermostat

The Berendsen thermostat is essentially a cross between the NVE and Gaussian algorithms that uses a user defined relaxation constant that determines the degree of rescaling of the velocities. The Berendsen thermostat can be used to push a system towards a desired temperature as opposed to enforcing it. The basic algorithm is detailed below:

$$\underline{v}(t + \frac{1}{2}\Delta t) \leftarrow \eta \left(\underline{v}(t - \frac{1}{2}\Delta t) + \Delta t \frac{\underline{f}(t)}{m} \right)$$

$$\underline{r}(t + \Delta t) \leftarrow \underline{r}(t) + \Delta t \underline{v}(t - \frac{1}{2}\Delta t)$$

$$\eta \leftarrow \sqrt{\left[1 + \frac{\Delta t}{\tau} \left(\frac{T_{ext}}{T} - 1 \right) \right]}$$

Where τ is the user defined time constant in units of *ps*. As with the Gaussian thermostat, the temperature is calculated self consistently over three iterations.

1.8.4 NVT with Nose-Hoover Thermostat

[to be added later]

1.8.5 Stochastic Boundary Conditions

To simulate coupling to a heat bath and to physically allow energy gained by the system from a moving tip to be carried off to infinity, all or a subset (at the boundaries) of the atoms in a system can be subject to a random impulse and frictional force at every timestep, governed by the Langevin equation:

$$\frac{\partial \underline{p}_i}{\partial t} = -\frac{\partial U}{\partial \underline{q}_i} - \gamma \underline{p}_i + F^+$$

Where p and q and the atomic momentums and positions respectively, U is the total system energy, γ is the parametric friction coefficient and F^+ is a Gaussian distributed random force, the size of which is given by the second fluctuation-dissipation theorem:

$$\langle F_i^+(t_1) F_j^+(t_2) \rangle = 2\gamma k_B T_{ext} \delta_{ij} \delta(t_1 - t_2)$$

The algorithm used to integrate the equations of motion is the same as the Verlet Leapfrog, except that the velocity proportional friction force and random impulse is added to the total systematic force prior to the integration. The larger the friction coefficient, γ , the faster the system will react to a change in temperature and as $\gamma \rightarrow 0$ the system tends to the NVE ensemble. The code allows the friction coefficient of each degree of freedom in the system to be assigned individually by the user in the input file.

It has been demonstrated in [19] that this scheme is only valid when the decay time of the friction coefficient is much larger than the time step in the simulation, i.e.

$$\Delta t \ll \frac{1}{\gamma}$$

The code will not prevent or warn the user from using any value for the friction coefficient.

1.8.6 Shells

The code has the facility to treat shells dynamically by one of two separate algorithms, which the user selects. The first of these the Dynamical Shell model, first introduced in [20], each shell is given a small mass and the shell motions are integrated along with the atomic cores, in a similar way to a di-atomic molecule. This method is computationally cheap, however a smaller timestep must be used to allow the high frequency core-shell oscillations to be integrated accurately. Also over long time periods, kinetic energy unphysically leaks in to the core-shell degree of freedom, although the program calculates the core-shell temperature and so this can be monitored.

The second algorithm, first introduced in [21], relaxes the zero-mass shells at each timestep by minimising the system energy with respect to shell positions. The shells are then removed for the integration, while the cores are still subject to the shell systematic force. This algorithm is more physical and accurate than the first, however it is much more computationally expensive, since a minimisation must take place at every timestep.

At the present time these dynamical shell models have only been implemented in the NVE ensemble.

2 Virtual AFM

2.1 How the virtual machine works

The virtual AFM was built on the basis of the precise experimental determination of the transfer function and impulse response of each element of the real AFM in order to reproduce as precisely as possible the experimental behaviours. A room temperature commercial AFM/STM [22] was used with its standard electronic control system, except for the frequency demodulator which was replaced by a digital Phase Locked Loop (PLL) based frequency demodulator [23]. The block diagram of the system is shown in figure 6.

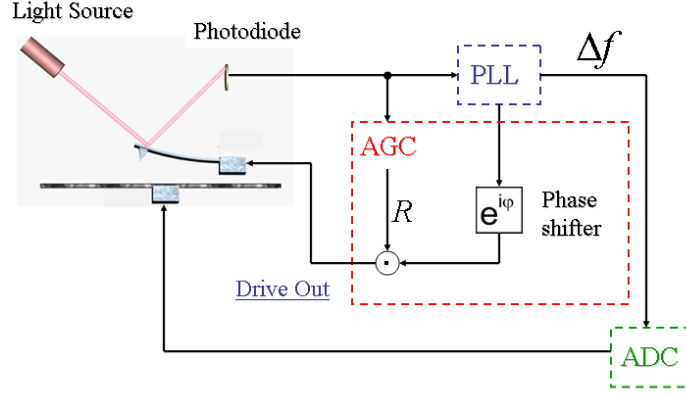


Figure 6: Schematic of the FM-AFM.

FM-AFM involves two control loops:

- The first one is supposed to maintain the cantilever oscillation amplitude at a preset value A_0 (typically from a few nm to tens of nm). This is achieved by an Automatic Gain Control block (AGC), which acts on the power delivered to the cantilever excitation piezoelectric ceramics. This power can be made proportional to the power dissipated by the tip-surface interaction by a proper setting of the phase of the excitation drive of the cantilever [24, 25, 26]. It is used to produce dissipation images.
- The second one is supposed to maintain the resonance frequency shift induced by the tip-surface interaction at a preset value Δf_c (typically from a few Hz to hundreds of Hz). This is achieved by an Automatic Distance Control block (ADC), which adjusts the tip-surface distance $D(t)$. It is used to produce "topographic" images.

2.1.1 Equation of motion of the cantilever

The dynamics of the cantilever is governed by the following equation of motion:

$$M\ddot{z}(t) + \frac{M\omega_0}{Q}\dot{z}(t) + kz(t) = RA_{pll}\sin(\omega_{pll}t + \varphi) + F_{int}(z(t) + D(t); \dot{z}(t)) \quad (30)$$

It is solved numerically using the velocity Verlet algorithm [27]. In this equation $z(t)$ is the vertical displacement of the cantilever around its mean value. The actual distance of the cantilever from the surface plane is given by $z(t) + D(t)$ as shown in Fig. 7.

Various terms in the above equation have the following meaning:

- Terms in the LHS correspond to a damped harmonic oscillator: $M = k/\omega_0^2$ is the effective cantilever mass, where k is cantilever stiffness and $\omega_0 = 2\pi f_0$ is the resonance frequency of the free cantilever; in addition, there is also a friction term in the LHS containing the dimensionless quality factor Q ; these are all intrinsic parameters of the probe.

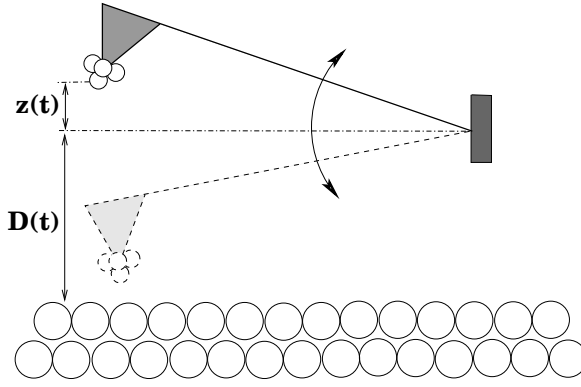


Figure 7: Schematic of the oscillating cantilever system.

- The first term in the RHS is the excitation signal which is used to maintain the cantilever oscillation amplitude at a preset value A_0 . The first factor in the excitation amplitude R is called a damping or dissipation term and is considered in section 2.1.2 in more detail. It's just a variable gain. Note that the other part of the signal, $A_{pll} \sin(\omega_{pll}t + \varphi)$, is produced by the PLL as described in more detail in sections 2.1.2 and 2.1.3. It drives the piezo of the cantilever.
- $F_{int}(z(t) + D(t); \dot{z}(t))$ corresponds to the conservative and dissipative interactions between the tip and the surface (the latter is not implemented presently).

2.1.2 Amplitude regulation

Amplitude regulation is achieved by the Automatic Gain Control (see figure 8). The modulus $A(t)$ of the cantilever oscillation $z(t)$ is extracted by a diode followed by a first-order low-pass filter, modeled by:

$$A(t) = \frac{\pi}{2\tau_D} e^{-t/\tau_D} \int_0^t |z(t')| e^{t'/\tau_D} dt' \quad (31)$$

with τ_D being the time constant of the envelope detection.

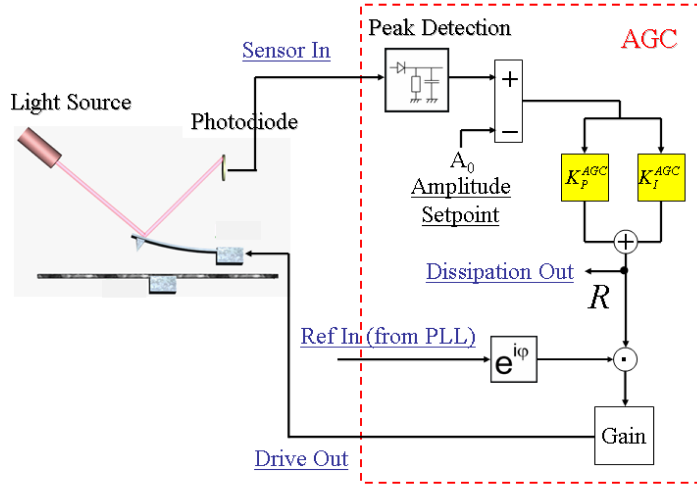


Figure 8: Automatic Gain Control.

$A(t)$ is then compared to the amplitude setpoint A_0 . The error signal $A(t) - A_0$, after being fed into a PI (Proportional-Integral) system [28, 29, 30], is used to change the variable gain $R(t)$. Both the Proportional gain K_P^{AGC} (controlling the “stiffness” of the AGC loop) and the Integration gain K_I^{AGC} (controlling the “viscosity” of the AGC loop) can be set by the user. One obtains the dynamic gain $R(t)$:

$$R(t) = K_P^{AGC} [A(t) - A_0] + K_I^{AGC} \int_{-\infty}^t [A(t') - A_0] dt' \quad (32)$$

Without tip-surface interaction, $R(D \rightarrow \infty) = \frac{k_c A_0}{Q_c}$ with $A_{pll} = 1$.

2.1.3 Frequency demodulation and cantilever excitation

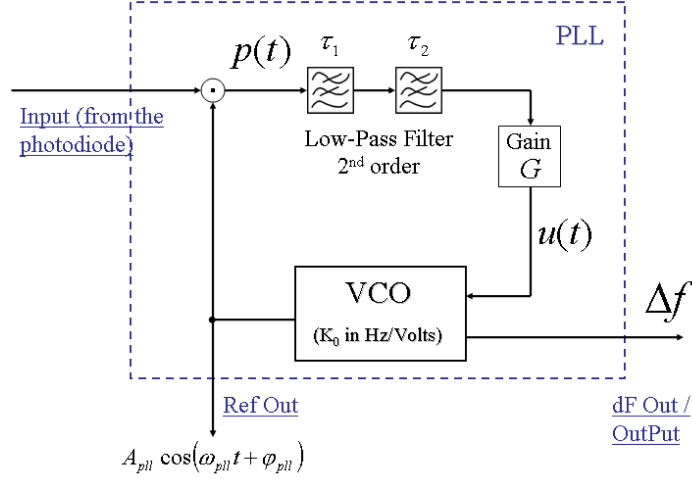


Figure 9: Phase Locked Loop

Far from any tip-surface interaction, the pulsation $\omega_{pll}(t)$ of the PLL (see figure 9) reference signal is set by the user to the free frequency of the cantilever such as $\omega_{pll}(t) = \omega_0$. The VCO (Voltage Controlled Oscillator), inside the PLL, produces this reference signal by the integration of the input voltage $u(t)$:

$$\omega_{pll}(t) t = \omega_0 t + \int_{-\infty}^t \Delta\omega(t') dt' = \omega_0 t + K_0 \int_{-\infty}^t u(t') dt' \quad (33)$$

where K_0 (in Hz) is the conversion factor (voltage to frequency) of the VCO [31].

The signal $u(t)$ which is proportional to the detuning, $\Delta\omega(t) = \omega(t) - \omega_0$, of the cantilever frequency, is obtained by low-pass filtering the product $p(t)$ of the reference signal by the cantilever signal $z(t)$ coming from the photodiode. Hence:

$$p(t) = z(t) A_{pll} \cos(\omega_{pll}(t) t + \varphi_{pll}) \quad (34)$$

In first approximation, if we consider $z(t) = A(t) \sin(\omega(t) t + \varphi)$, one obtains:

$$p(t) = \frac{A_{limited} A_{pll}}{2} [\sin((\omega(t) - \omega_{pll}(t)) t + (\varphi - \varphi_{pll})) + \sin((\omega(t) + \omega_{pll}(t)) t + (\varphi + \varphi_{pll}))] \quad (35)$$

with a limiter circuit, at the input of the PLL, that limits the signal amplitude $A(t)$ to a constant level $A_{limited}$. This signal $p(t)$ then goes through a second order low-pass filter defined by two time constants τ_1 and τ_2 :

$$p_1(t) = \left[\frac{1}{\tau_1} \int_{-\infty}^t p(t') \exp\left(\frac{t'}{\tau_1}\right) dt' \right] \exp\left(\frac{-t}{\tau_1}\right) \quad (36)$$

$$u(t) = G \left[\frac{1}{\tau_2} \int_{-\infty}^t p_1(t') \exp\left(\frac{t'}{\tau_2}\right) dt' \right] \exp\left(\frac{-t}{\tau_2}\right) \quad (37)$$

Thanks to this low-pass filter, the low frequency part, $\omega(t) - \omega_{pll}(t)$, of the signal $p(t)$ is extracted. The parameter G allows to adjust the loop gain of the PLL in order to find the good stability-precision compromise [32]. Hence:

$$u(t) = G \frac{A_{limited} A_{pll}}{2} \sin((\omega(t) - \omega_{pll}(t)) t + (\varphi - \varphi_{pll})) \quad (38)$$

When $\omega_{pll}(t) = \omega(t)$, the PLL is locked on the cantilever frequency. So, one can write, in the case of small changes $\Delta\varphi = \varphi - \varphi_{pll}$:

$$u(t) \simeq G \frac{A_{limited} A_{pll}}{2} \Delta\varphi \quad (39)$$

giving the frequency detuning $\Delta\omega(t) = K_0 u(t)$.

2.1.4 Distance regulation

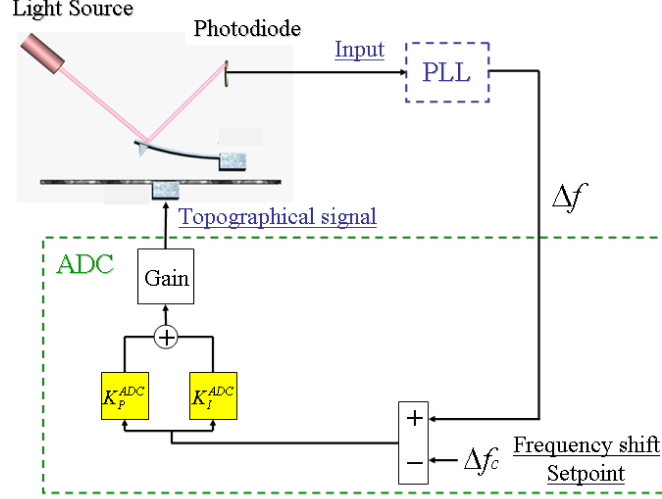


Figure 10: Automatic Distance Control.

The $\Delta f(t) = \frac{\Delta\omega(t)}{2\pi}$ signal coming from the frequency demodulator (see figure 9) is used to regulate the distance $D(t)$ between the cantilever and the surface in order to satisfy the frequency setpoint Δf_c (see figure 10) defined by the user [33, 34]. Following this rule, one gets the following expression:

$$D(t) = D_1 + K_P^{ADC} [\Delta f(t) - \Delta f_c] + K_I^{ADC} \int_{-\infty}^t [\Delta f(t') - \Delta f_c] dt' \quad (40)$$

where we find, like in the AGC (see equation 32), a PI (Proportional-Integral) system with gains K_P^{ADC} and K_I^{ADC} defined by the user. D_1 is an adjustable offset in distance.

2.2 Influence of experimental noises on AFM images

Another useful application of the virtual AFM is the realistic evaluation of the limitations imposed by the noise generated by the components of the system. Sources of noise in beam deflection AFM have been discussed in [35]. The major ones are [36]:

The thermal noise of the cantilever [37, 33], whose power spectral density reads:

$$S_F(f) = \frac{4 k_B T k}{2\pi f_0 Q} \quad \text{in } N^2.Hz^{-1} \quad (41)$$

with the temperature T , the Boltzmann constant k_B . In virtual AFM, this noise is implemented at left in equation 30 as a noise force.

The amplitude shot noise [38, 36] of one photodiode quadrant detector:

$$S_A^{shot-noise}(f) = 2 q_e V_{ph} R_{trans} G_{opt}^2 \quad \text{in } m^2.Hz^{-1} \quad (42)$$

with the elementary charge q_e , the photodiode voltage V_{ph} (1 Volt in our experimental conditions), the transimpedance resistor R_{trans} (500 k Ω [22]). G_{opt} is a factor to convert the cantilever oscillation in volts coming from the photodiode in nanometers. It depends of the geometrical design of the optical detection but also of the kind of light source used (Laser, LED, ...). In our case, we measured $G_{opt} = 400 \text{ nm.V}^{-1}$.

Finally, the Johnson noise of the resistor of the current-voltage converter that monitors the photodiodes:

$$S_A^{Johnson}(f) = 4 k_B T R_{trans} G_{opt}^2 \quad \text{in } m^2.Hz^{-1} \quad (43)$$

The amplitude shot noise and Johnson noise are both used as amplitude noises inside $z(t)$ the vertical displacement of the cantilever coming from the photodiode signal.

2.3 How one can find parameters of the virtual AFM

The aim of the virtual machine is to mimic the dynamic of a true experimental AFM system. In this subsection we shall briefly describe a response of the whole machine (Omicron RT-AFM with an easyPLL Nanosurf Demodulator) on a square periodic offset added to the distance signal $D(t)$. This is an example of how one can experimentally find the correct parameters needed to run the virtual AFM machine.

Fig. 11 (a) shows the frequency response to a periodic square excitation added to the surface-tip distance D while the microscope was in operation above a MoS₂ substrate covered by silver clusters. This measurement was performed for two different settings of the ADC feedback loop. A loop gain of 2% (slow response) corresponds to the typical value for normal imaging while a loop gain of 20% (fast response) is only used when approaching the surface at the beginning of the experiment. The asymmetry of the response between a positive and negative Δf (the positive peak is smaller and wider than the negative one) is a consequence of the increase of the absolute value of the slope of the van der Waals interaction when the tip approaches the surface.

Fig. 11 (b) shows the response calculated with the virtual AFM modelling the tip by a sphere of the radius 20 nm and taking into account only the van der Waals interaction between the tip and sample, with a Hamaker constant of 1 eV. The experimental behaviour can be well reproduced by choosing the parameters K_P^{ADC} and K_I^{ADC} [30]. The shape of the peaks is qualitatively correct while the time constants, ranging from a few milliseconds at 20% to a few tens of milliseconds at 2%, are of the same order of magnitude as in the experiment.

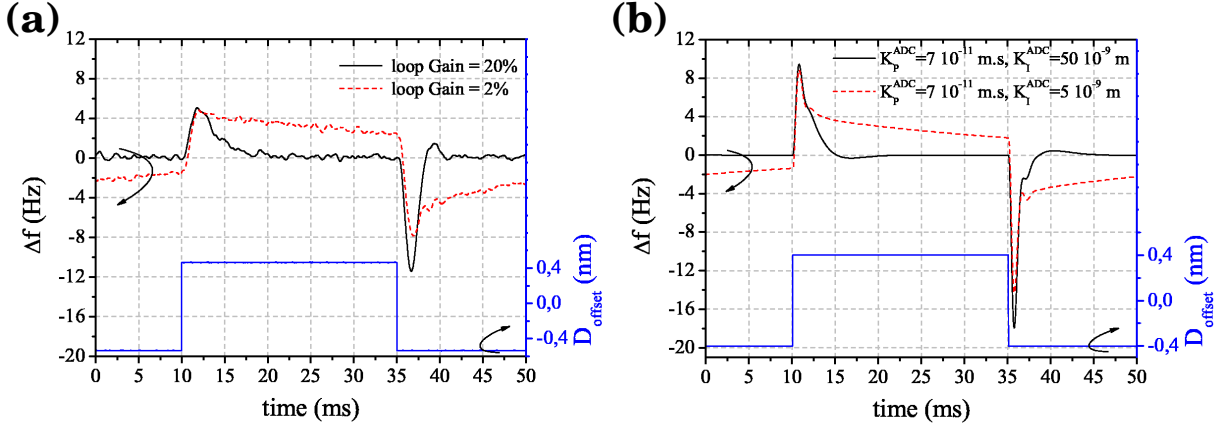


Figure 11: Experimental (a) and calculated (b) response of the ADC. Electrostatic forces have been compensated following Ref. [39].

Part II

Installation

3 How to compile the code

The code should be in the form of a gzipped tarball, such as `scifi_4.3.5.tgz`. Assuming this, then type:

```
tar -zxvf scifi_4.3.5.tgz
```

this will extract all the necessary files, including the main of the code (including the vAFM part) and **IMAGE** (the image part of the code), a directory **examples** containing some examples input files and the corresponding output files (not all may be up-to-date and may require editing).

To compile the program, a **Makefile** in either of the main directory and in **IMAGE**, should be edited to include the command of your Fortran 77 or Fortran 90 compiler, along with any compiler options.

The code can be compiled into either a serial executable or a MPICH parallel version, via the **cpp** precompiler. To compile a parallel version, the **MPIFLAG = -DMPI** must be added to the **Makefile**, as well as the MPICH flags to your compiler and the path of your **mpich.h** header file.

To compile the code, type

```
make
```

in either of the two directories; this will produce the executables **shellm** and **image**, respectively. Make sure that you have access to both executables, i.e. that the directories are included in the **\$path** variable.

You can now run the code (see below how). Make sure that you can reproduce the output from the the example inputs provided in the **examples** directory (section V).

4 parameter file

The file **param.inc** in the main directory contains all the matrix sizes for the calculation. This file must be edited before compilation if changes are needed. Below is an example:

```
implicit real*8 (a-h,o-z)
c..... parameters:
c NOKD - max number of species
c NOLT - max number of atoms
c NOCH - max number of all charges (cores+shells)
c NOPAR - number of exp/power terms in the short-range interaction
c NOFR=max_free - max number of degrees of freedom for optimisation
c isaved_step - the MD restart saving interval
c max_proc - maximum number of processors
c NOBD - maximum number of covalent bonds per atom

parameter ( NOPAR=5,NOKD=10, NOLT=4500, NOCH=2*NOLT )
parameter ( NOKD1=NOKD*(NOKD+1)/2 )
parameter ( NOFR=6300 , max_free=NOFR )
c BFGS_st_desc_max - max number of times BFGS can run steepest descent
c in a row. This prevents the code to get stuck in this.
parameter ( iBFGS_st_desc_max=1 )
parameter ( isaved_rst = 100 )
parameter ( NOBD = 6 )
parameter ( max_proc = 100 )
c NOCNSTR - max number of constraints
c NinvAt0 - max number of atoms participating in every constraint parameter (NOCNSTR=10,NinvAt0=10)
```

These are the default values, but they can be reduced to reduce memory demands or increased for bigger systems. The most crucial parameter is **NOLT** which controls the maximum number of charges (**NOCH = 2*NOLT**) in the system, if this is changed then **NOFR**, the maximum degrees of freedom, should be changed correspondingly.

5 virtual_param.inc file

The file `virtual_param.inc` contains all the matrix sizes for the calculation. This file must be edited and a compilation performed if changes are needed. Below is an example:

```
parameter (pi=3.141592653589793d0, twopi=2.d0*pi)
c..... max number of steps/stages
parameter (N_of_stages_max=99)
c..... scans: max number of scans allowed
parameter (N_of_scans_max=5)
c..... units: distance, force double precision m_A,N_eVpA,J_eV
parameter (m_A=1.0d10,N_eVpA=6.2415d8,J_eV=6.2415d18)
c..... threshold for the distance
parameter (threshold=1.0d-10)
```

The most important parameters are:

- `N_of_stages_max` - maximum number of elementary stages for the whole run
- `N_of_scans_max` - maximum number of scans allowed during a single run

6 How to run the code

The code is run with a command line like this:

```
shellm name [option]
```

where `name` is the job identifier. Only a single option is available which is the word `restart`. This is used to restart the calculation from the file `name.RST` written previously.

Part III

Input

7 General

The code reads in a single unformatted input file which can be in one of two forms: either an initial file (**name.INN**) or a restart file (**name.RST**). The only difference is that a restart file has been produced by the code itself and normally contains relaxed positions of atoms and shell-polarization. A restart file will automatically be produced after every run of the code, regardless of the options selected.

The code will use an initial (**.INN**) file of the appropriate name if no **option** is given. If the keyword 'restart' is given as an option then the program will open the restart file of the appropriate name as input. The code will also produce an input file **input.img** if the **.<IMAGE>** box is present. This contains all atomic positions (excluding **frozen-tip**), charges and general information about the system relevant to the image calculation, and can be ignored except for debugging purposes. The image part of the code runs independently of the main code, but is called automatically without requiring user intervention.

The input file itself is separated into specific boxes for each category of input parameters. Each box begins with the keyword name of the box in the format **.<keyword>** and then the appropriate keywords and parameters for that box are listed. Keywords must be placed in the input box they are classed in, they will be ignored in other boxes. Some keywords require another controlling word or parameter value, whereas others are activated by just being present. After the example file, each box and its keywords are explained, along with possible parameters which can be used.

Comments can be entered into the input file by placing **##** before the line. Empty lines will be ignored. There is no specific order required for the boxes in the input file, but there is some order needed for specific keywords within the boxes. These position dependent keywords will be highlighted below.

Boxes described in Section 8 correspond to the SciFi part of the code. If an error is generated when reading those boxes (in particular, if some or all boxes are missing), then the SciFi part is switched off. Next, an vAFM input is read in. If there is no error, the vAFM part of the code is working relying on an existing force field. Otherwise, the code stops with an error message. Conversely, if there is only an error in the vAFM part, then the code works as a SciFi code. The vAFM input part is described separately in Section 9.

Also note that chemically identical atoms, but belonging to different groups of species, should be given different identifier; the 3rd character may also be used (e.g. C1, C2, Mg1, Mg5, etc.).

8 SciFi input

8.1 Example Input File

This input file contains a system where the tip consists of only one atom at the end of a conducting sphere interacting with a small defected cluster on top of a conducting substrate.

```
.<Toleranc>
do_md
printing tiny
shells frozen
xyz write
##g98-file write
max_opt    1
optim bfgs
en_tol 0.150000E-05
es_tol 0.500000E-01
frc_tol 0.100000E-01
crd_tol 0.500000E-01
##test 0.0001
.<End>

.<MDcontrol>
numsteps 2000
tstep 0.00001
dtemp 330.000
algorithm nve
```

```

shells dynamic
tau 0.2000
statistics 1
trajectory 20
traj_fmt 1
vel_key 0
.<End>

.<ShellPar>
  0 0.400000E-01 -.204000E+01 0.143000E+02
Mg 0.200000E+01 0.000000E+00 0.000000E+00
Cr 0.300000E+01 0.000000E+00 0.000000E+00
.<End>

.<Coord>
##move 1.0 1.0 1.0
##rotate 90.0 90.0 90.0
##centre 5.0 5.0 5.0
##scan 0.2

tip-atoms
Cr 0.00000 0.00000 5.50000 1 1 1
cluster-atoms
Cr 0.00000 0.00000 0.00000 1 1 1
O -2.12200 0.00000 0.00000 1 1 1
O 0.00000 -2.12200 0.00000 1 1 1
O 0.00000 0.00000 -2.12200 1 1 1
O 0.00000 2.12200 0.00000 1 1 1
O 2.12200 0.00000 0.00000 1 1 1
Mg -2.12200 -2.12200 0.00000 1 1 1
Mg -2.12200 2.12200 0.00000 1 1 1
Mg 2.12200 -2.12200 0.00000 1 1 1
Mg 2.12200 2.12200 0.00000 1 1 1
Mg -4.24400 0.00000 0.00000 1 1 1
Mg 0.00000 -4.24400 0.00000 1 1 1
Mg 0.00000 4.24400 0.00000 1 1 1
Mg 4.24400 0.00000 0.00000 1 1 1
Mg 0.00000 -2.12200 -2.12200 1 1 1
Mg 0.00000 2.12200 -2.12200 1 1 1
Mg -2.12200 0.00000 -2.12200 1 1 1
Mg 2.12200 0.00000 -2.12200 1 1 1
Mg 0.00000 0.00000 -4.24400 1 1 1
O -2.12200 -2.12200 -2.12200 1 1 0
O -2.12200 2.12200 -2.12200 1 1 0
O 2.12200 -2.12200 -2.12200 1 1 0
O 2.12200 2.12200 -2.12200 1 1 0
O -4.24400 -2.12200 0.00000 1 1 0
O -4.24400 0.00000 -2.12200 1 1 0
O -4.24400 2.12200 0.00000 0 0 0
O -2.12200 -4.24400 0.00000 0 0 0
O -2.12200 0.00000 -4.24400 0 0 0
O -2.12200 4.24400 0.00000 0 0 0
O 0.00000 -4.24400 -2.12200 0 0 0
.<End>

.<BondToleranc>
O O 0.000000000000000 0.000000000000000
Mg O 0.000000000000000 0.000000000000000
Mg Mg 0.000000000000000 0.000000000000000
Cr O 0.000000000000000 0.000000000000000
Mg Cr 0.000000000000000 0.000000000000000
Cr Cr 0.000000000000000 0.000000000000000
.<End>

.<PairPotenc>
cutoff
  0 0 0.600000E+01
    exp 1
      9547.900000000000 0.219160000000000
    pow 1
      -32.8000000000000 6.00000000000000

```

```

Mg  0  0.600000E+01
    exp  1
    1284.380000000000      0.2996900000000000
    pow  0
Mg  Mg  0.100000E+00
    exp  0
    pow  0
Cr   0  0.600000E+01
    exp  1
    1284.380000000000      0.2996900000000000
    pow  0
Cr  Mg  0.100000E+00
    exp  0
    pow  0
Cr  Cr  0.100000E+00
    exp  0
    pow  0
.<End>

```

```

.<Image>
plane
sphere
decay
qwrite
printing tiny
precis      15
Z_plane -.110000E+02
bottom_sph 0.000000E+00 0.000000E+00 5.0
bias 0.100000E+01
radius 0.100000E+03
smooth 0.3
buffer 2.0
.<End>

```

8.2 Tolerance Box

Box Keyword: `.<Toleranc>`

This box controls the type and amount of output produced by the program, and also the accuracy of the values calculated. It uses the following keywords which can be positioned independently within the box:

8.2.1 General settings

write_energy_forces

If present energies, positions and forces of all atoms will be dumped in the file `job.31` every time they are calculated, where `job` is the name/job identifier. May be found useful in testing this or other codes.

Default: **off**

Example: **write_energy_forces**

write_force

If present forces on all frozen tip atoms (regions “frozen tip” and “tip buffer”) will be written to a file `filename` specified. This can be used to reduce the noise of the tip force by previously relaxing the tip alone and writing any residual forces left (see the next option) to the file.

Options: **filename**

Default: **off**

Example: **write_forces myfile**

read_force

If present forces on all frozen tip atoms written to a file **filename** (e.g. for an isolated tip) will be read in and subtracted from the current tip force to reduce the noise (see the previous option).

Options: **filename**

Default: off

Example: **read_forces myfile**

do_lateral

If present, the lateral forces on the tip (in the x and y directions) will also be calculated and dumped in files **name.TFX** and **name.TFY** alongside with the z force in **name.TFZ**. As a default, only the z force on the tip is calculated. Note that presently, only the z -force can be calculated if Image is on, so that the code will stop if do_lateral is chosen together with the Image box.

Options: none

Default: off

Example: **do_lateral**

shells

Controls whether shells in regions “cluster atoms” and “tip atoms” are allowed to relax.

Options:

- **moveall** - shells of all atoms are allowed to relax; this option is useful, for instance, if one wants to calculate the electronic polarisation alone keeping all atoms in their initial positions;
- **frozenall** - all shells are frozen irrespective of whether cores are allowed to move or not; this would correspond to the *frozen core approximation* if initial positions of shells are the same as for their cores;
- **follow** - only shells of frozen cores are frozen; shells of atoms allowed to move will follow them; this is likely to be the most useful and frequently used option.

Default: **moveall**

Example: **shells moveall**

8.2.2 Output control

printing

Controls amount of information in output file.

Options: **no**, **tiny**, **small**, **medium**, **large**, **huge**

Default: **tiny**

Example: **printing medium**

xyz

If present, a xyz format file with all atomic coordinates after run finishes will be printed out. This will produce two files, **name_core.xyz** and **name_shll.xyz** containing the atomic and shell positions, respectively.

Options: none

Default: off

Example: **xyz**

movie

Flag to produce a cumulative xyz file during relaxation for use in producing a movie of the displacements with appropriate software e.g. **xmol**.

Options: none

Default: off

Example: **movie**

g98

If present, a Gaussian98 format file after run finishes will be printed out.

Options: none

Default: off

Example: **g98**

update_RST

If present, this flag will activate an update of the atomic positions in the RST file after every relaxation step if minimising energy, and will update atomic positions and velocities after every `i_saved_RST` timesteps (given in `param.inc` file) during an MD run. This will slow down the code; however, it will save you the latest geometry in the case of a crash or if you want to kill a running job.

Options: none

Default: off

Example: **update_RST**

8.2.3 Geometry optimisation options

max_opt

Maximum number of relaxation iterations (N) to be used within the optimisation method chosen (see below).

Options: N

Default: N = **100**

Example: **max_opt 15**

optim

The energy minimisation which is specified using this keyword is the default algorithm. Note that if the `.<MDcontrol>` box is found (section 8.7.1), then the Molecular Dynamic simulation is performed in which case this option is either ignored when fictitious dynamics with shells is assumed, or used to optimise shells positions in the case of the only-cores dynamics (more details in section 8.7.1).

Controls the relaxation algorithm used, at present either steepest descent, bfgs, conjugate gradient or both. In the latter case the conjugate gradient method is applied during the first 3 iterations after which the bfgs is applied to finish the relaxation calculation. To use both bfgs and conjugate gradient just put the `optim` flag twice in the input, once with `bfgs` and once with `conj`. The `conj` option can also take an additional parameter which changes the default conjugate gradient algorithm from `flreeves` to `polak_rib`.

Options: **steep**, **bfgs**, **conj** (**polak** or **flreeves** (*default*))

Default: off

Example 1: **optim conj polak**

Example 2: **optim conj**

Example 3: **optim bfgs**

Example 4: **optim bfgs**

optim conj

en_tol

Tolerance of the total energy in eV (#).

Options: #

Default: # = **1.0e⁻⁶**

Example: **en_tol 1.0e⁻⁵**

crd_tol

Tolerance of the coordinates in Å(#).

Options: #

Default: # = **0.01**

Example: **crd_tol 0.001**

frc_tol

Tolerance of the forces on the atoms in eV/Å(#).

Options: #

Default: # = **0.1**

Example: **frc_tol 0.01**

es_tol

Maximum energy change during 1-dimensional search in eV (#) during steepest descent (**steep**) or conjugate gradient (**conj**) methods.

Options: #

Default: # = **0.05**

Example: **es_tol 0.1**

shake

If present the atom which has the maximum force is moved by the step # (in Å) along the force if either of the optimisation routines (BFGS or CG) got stuck. Altogether this can be done only 2 times during the optimisation. If the optimisation engine got stuck for the 3rd time, the code will stop.

Options: #

Default: # = **0.01**

Example: **shake 0.1**

8.2.4 Testing of the code

test

This flag will activate a numerical test of the input, the energy, the forces on the atoms or the tip depending on the option selected. A value for the step # used in calculating the numerical force can also be given.

Options:

- **direct** (#1 #2 #3 #4) - energy along a single direction (= degree of freedom) #1, between values of #2 and #3, is calculated using step #; file **fort.19** will contain the results; *default* used: #1 =1, #2 =-0.5, #3 = 0.5, #4 = 0.01;
- **atom** (#) - compares analytical and numerical forces for all degrees of freedom in the system; file **fort.19** will contain a comparison of the forces on each atom and the error;
- **tip** (#) - compares the analytical and numerical vertical force on the tip; file **fort.19** will contain a comparison of the forces and the error. Note that **max_opt** must be greater than one for this option to work;
- **input** - will check the input and the code will then stop.

Default (general):

- all test options are off;
- the value for the step # = **0.0001**;

Example 1: **test atom 0.00005**

Example 2: **test tip 0.01**

Example 3: **test input**

8.3 Species

8.3.1 ShellPar Box

Box Keyword: .<ShellPar>

This box is used to input all species identifiers used in the current calculation, alongside the charges and masses on atoms and their shells (in electron charge and atomic mass units respectively), along with the spring constant between them (in Newton metres). It has the following format:

```

Element1  $M_{core}$   $Q_{core}$   $M_{shell}$   $Q_{shell}$   $k$ 
Element2  $M_{core}$   $Q_{core}$   $M_{shell}$   $Q_{shell}$   $k$ 
etc ....

```

where Element1, Element2, etc. are the atomic element names from the periodic table. The element name can also include a numerical identifier as a third character, e.g. Mg1, to differentiate between the same atom species with different interactions. This can be used to model atoms of the same species in the surface and tip differently.

Any atoms which have both a Q_{core} and Q_{shell} will have a shell created automatically and positioned at the same place as the initial atom coordinates. This will then be free to relax independently of the original atom to represent electronic polarization. If Q_{shell} is zero then no shell will exist for this element. If an energy minimisation is being performed, then the values provided for the core and shell masses will be ignored.

During an MD simulation shell dynamics is handled by one of two algorithms, specified in the MDcontrol box, either the fictitious dynamic shell model or the relaxed shell model (sections 8.2.3 and 8.7.1). In the first of these, any shells must be given a (usually small) mass, so that equations of motion can be integrated. In the second, shells are relaxed at every timestep and require no mass (if a mass is provided it will be ignored).

8.3.2 Species_Types

Box Keyword: .<SpeciesTypes>

This box is mandatory if either Tersoff or many-body interactions are present. It is used to identify species belonging to Tersoff group of atoms as well as many-body group of atoms. Either one of the following two lines, or both of them, should be present:

```

Tersoff_Species S1 S2 ....
ManyBody_Species T1 T2 ...

```

where S1, S2, etc. are all species (any order) corresponding to the Tersoff group of species (e.g. Si, Si#), and T1, T2, etc. are the many-body species. Note that in the case of the Tersoff species the boundary species (with the # at the end) must also be present as a separate species.

8.4 Box with atomic coordinates

Box Keyword: .<Coord>

Atomic positions are specified in this box.

8.4.1 move

This flag moves the tip by the given x, y, z values in the x, y, z directions respectively prior to relaxation. It must appear before the coordinates of the atoms. Note that this also moves the conducting sphere, if present, by the same amount.

```

Options:  $x$   $y$   $z$ 
Default: off,  $x=0.0$ ,  $y=0.0$ ,  $z=0.0$ 
Example: move 0.0 -4.0 15.0

```

8.4.2 rotate

This flag rotates the tip around the x, y, z axes by the angles α , β , γ (in degrees) respectively prior to relaxation. It must appear before the coordinates of the atoms.

```

Options:  $\alpha$   $\beta$   $\gamma$ 
Default: off,  $\alpha = \beta = \gamma = 0$ 
Example: rotate 0.0 0.0 45.0

```

8.4.3 centre

This flag is used to specify the centre of the rotations given after the **rotate** keyword. It must appear before the coordinates of the atoms.

```

Options:  $x$   $y$   $z$ 
Default: off,  $x=0.0$ ,  $y=0.0$ ,  $z=0.0$ 
Example: centre 10.0 10.0 0.0

```

8.4.4 scan

This keyword activates the script-based “scanning” mode, when only a **move** option with a specific displacement (x, y, z) is added to the final **name.RST** file after relaxation. This can be used in a shell script to run a series of **name.RST** files and produce a plot of energy or force vs. distance, with the designated tip atoms moving the specified step in each file.

Options: #

Default: off

Example: **scan 0.1 0.1 0.1**

Since version 3.11 this option became redundant after introducing the dynamic tip option which does the same job during one job run and thus avoids using complicated shell scripts (see section 8.8). However, it can still be used if desired!

8.4.5 Atomic Coordinates

The rest of the coordinates input box is used to input the positions of the atoms in the system in cartesian coordinates. Every atom is entered in the same format, but each belongs to one of five classes of atoms depending on the keyword used at the beginning of its group. The five keywords are:

- **cluster-atoms** - relaxed, contributes to potential and image interactions. Usually used to represent relaxed atoms in a cluster.
- **cluster-buffer** - frozen, contributes to potential and image interactions. Usually used to represent frozen atoms in a cluster.
- **tip-atoms** - relaxed, contributes to potential and image interactions. Included in all operations on the tip (**move**, **rotate**). Usually used to represent relaxed atoms in the tip.
- **tip-buffer** - frozen, contributes to potential and image interactions. Included in all operations on the tip (**move**, **rotate**). Usually used to represent frozen atoms in the tip.
- **frozen-tip** - frozen, contributes to potential interactions, but is removed from image calculations. Included in all operations on the tip (**move**, **rotate**). Usually used to represent frozen atoms in the tip which are within the conducting sphere and are forbidden from contributing to the image interaction.

These groups can only appear once each in the coordinates section, although the order in which they appear is irrelevant. The format of entry of the coordinates is as follows:

```
Element1 x y z (iRx iRy iRz)
Element2 x y z (iRx iRy iRz)
etc.....
```

The parameters **iRx**, **iRy**, **iRz** control whether an atom is allowed to relax in the x, y, z directions respectively - 0 0 0 would fix the atom in all three directions. They are optional and do not need to be specified. The default values for **iRx**, **iRy**, **iRz** are taken from the group the atoms belong to e.g. all **tip**- and **cluster-atoms** atoms are relaxed by default. The relaxation flags for shells are obtained appropriately from the **shell** option (section 8.2.1).

8.5 Interactions between atoms

8.5.1 PairPotenc Box

Box Keyword: .<PairPotenc>

This box contains the parameters which control the interactions (excluding electrostatic charge-charge interactions which are calculated implicitly) between atoms in the system. It must contain an entry for each pair of elements including itself. If a pair is entered twice, only the first pair will be used. The following general short-range interaction can be used:

$$V(r) = \sum_i C_i r^{-n_i} + \sum_j A_j e^{-\frac{r}{\rho_j}} \quad (44)$$

If you wish to use a cutoff (in Å) for the interactions, then the keyword **cutoff** must be the first entry in the pairpotenc box. If this keyword is not present then the cutoffs entered in the pair interactions will be ignored (in other words you do not need to enter any) and the interaction will be across the entire system. However, an error will be produced if the **cutoff** keyword is used, but cutoff values for every pair are not present. The parameters of the one pair's interaction are entered in the following format:

```
Element1 Element2 (cutoff)
exp #
 $A_j \rho_j$ 
pow #
 $C_i n_i$ 
```

The number # after **exp** and **pow** specifies the number of exponential and power terms in the interaction respectively. The appropriate number of parameters must then follow below, remembering that all values should be entered in units of Å of distance and eV of energy.

8.5.2 BondTolerance Box

Box Keyword: .<BondToleranc>

This box contains a list of each pair of elements in the system, with a maximum and minimum covalent bond length (in Å) which is used to create a connectivity table. It is mandatory of either (or both) Tersoff or many-body interactions are used. If the initial distance between two atoms falls within the tolerances, then a bond will be created between the two atoms. The connectivity table is then used to create lists of all bonds, angles, torsions and out-of-plane interactions, the parameters of which are provided in the relevant boxes.

```
Element1 Element2 Distmin Distmax
```

The code excludes pairs of atoms involved in bond and three body interactions from electrostatic and short range interactions.

8.5.3 quadratic_bond Box

Box Keyword: .<quadratic_bond>

This box lists every pair of elements interacting with a quadratic covalent bond, via the potential:

$$V_{ij} = V_0(r - r_0)^2$$

In the following format:

```
Element1 Element2  $V_0$   $r_0$ 
```

8.5.4 quadratic_angle Box

Box Keyword: .<quadratic_angle>

This box lists every sequence of three elements interacting with a quadratic angle potential:

$$V_{ij} = V_0(\theta - \theta_0)^2$$

In the following format:

```
Element1 Element2 Element3  $V_0$   $\theta_0$ 
```

8.5.5 torsion_1 Box

Box Keyword: .<torsion_1>

This box lists every sequence of four elements interacting with a torsional potential:

$$V_{ij} = V_0[1 + \cos(n\phi - \phi_0)]$$

In the following format:

Element1 Element2 Element3 Element4 V_0 ϕ_0 n

The program also understands the use of wild cards (*) in torsion sequences, i.e. when the potential is independent of the two end atoms:

* Element2 Element3 * V_0 ϕ_0

If the entire four atom torsion potential is present for an interaction it is used, if it is not present the the program searches for the appropriate potential with wild cards.

8.5.6 out-of-plane Box

Box Keyword: .<out_of_plane>

This box lists all elements forming an out-of-plane interaction with a potential:

$$V_{ij} = V_0[1 + \cos(n\chi - \chi_0)]$$

The angle χ is defined as the average of the three improper torsions formed by the interaction. The potentials are listed in the following format:

Element1 Element2 Element3 Element4 V_0 χ_0 n

Element2 is the central atom, and so the order of Element1, Element3 and Element4 is irrelevant.

8.5.7 Tersoff Box

Keyword: .<Tersoff>

Then, the following commands may be specified:

```
how_often_nn_update #
system PBC
```

where the first command specifies how often the connectivity table should be updated during an MD run (# is the number of MD steps after which the table is to be updated).

The the second command specifies the type of the periodic boundary conditions: PBC stands for either 'surf' or 'bulk' which is 2D and 3D, respectively. If this command is missing, the simulation system represents a finite cluster.

Default parameter of the Tersoff potential can be replaced in this box as well. There are parameters depending on one, two and three species. Not all of the may be given, onnly those which need replacing.

- One-species parameters. These (in order) correspond to the quantities β_i , c_i , d_i , h_i , n_i of Section 1.6.3 for species Sp :

```
- beta_ters Sp #
- c_ters Sp #
- d_ters Sp #
- h_ters Sp #
- n_ters Sp #
```

- Two-species parameters. These (in order) correspond to A_{ij} , B_{ij} , R_{ij} , D_{ij} , $\lambda_{ij}^{(1)}$, $\lambda_{ij}^{(2)}$ and χ_{ij} of Section 1.6.3 for a pair of species $Sp1$ and $Sp2$ (the parameters are all symmetrical with respect to the permutation of i and j , i.e. only one pair need to be given):

```
- A_ters Sp1 Sp2 #
- B_ters Sp1 Sp2 #
- R_ters Sp1 Sp2 #
- D_ters Sp1 Sp2 #
```

- `lambda1_ters Sp1 Sp2 #`
- `lambda2_ters Sp1 Sp2 #`
- `xi_ters Sp1 Sp2 #`
- Three-species parameter. This corresponds to $\lambda_{ijk}^{(3)}$ of Section 1.6.3 for a triple of species *Sp1*, *Sp2* and *Sp3* (all triples of species should be given on separate lines if needed, no permutational symmetry here):
- `lambda3_ters Sp1 Sp2 Sp3 #`

8.6 Lattice vectors

If Tersoff atoms are present and the **system** command specifies either 'surface' or 'bulk', then lattice vectors should be specified as well. This is done in this box.

Keyword: `.<Lattice>`

Then, either two ('surface') or three ('bulk') lattice vectors follow one on each line:

```
A1x A1y A1z
A2x A2y A2z
A3x A3y A3z      (only needed if 3D PBC)
```

8.7 Molecular Dynamics boxes

8.7.1 MDcontrol Box

Box Keyword: `.<MDcontrol>`

If present the program performs a Molecular Dynamics simulation on the system with the parameters given in the `.<MDcontrol>` box. If absent the program performs an energy minimisation (*default*). The corresponding input parameters are described below.

numsteps

Specifies the number of time steps to run the MD simulation for.

Options: #

Default: 1000

Example: **numsteps** 1000

tstep

Specifies the length of the timestep in picoseconds.

Options: #

Default: 0.001

Example: **tstep** 0.001

dtemp

Specifies the required system temperature in degrees Kelvin. It is used both to generate initial velocities (if they are not provided) and also to control the temperature if a thermostat (see below) is used.

Options: #

Default: 300.0

Example: **dtemp** 300.0

algorithm

Specifies the algorithm used to integrate the equations of motion.

Options:

- **nve** - Verlet Leapfrog
- **ber_nvt** - Verlet Leapfrog with Berendsen thermostat

- **evn_nvt** - Verlet Leapfrog with Gaussian thermostat
- **hov_nvt** - Verlet Leapfrog with Nose-Hoover thermostat
- **stochastic** - Verlet Leapfrog with atoms subject to stochastic thermostat. If this option is selected, friction coefficients must be provided in the `.<Friction>` box (section 8.7.3).

Default: **nve**

Example: **algorithm stochastic**

shells

Specifies the type of shell model to be used in dynamical simulations.

Options:

- **dynamic** - fictitious shells dynamics will be used: shells given mass and are moved together with cores;
- **relax** - only cores are dynamically moved whereas shells are relaxed at every timestep

Default: **dynamic**

Example: **shells dynamic**

tau

Specifies relaxation time in picoseconds for thermostats.

Options: #

Default: 1

Example: **tau 0.5**

statistics

Specifies the interval (in timesteps) between printing system information to the `name.STS` file.

Options: #

Default: 1

Example: **statistics 100**

trajectory

Specifies the interval (in timesteps) between printing configuration information to the `name.TRJ` file.

Options: #

Default: 1

Example: **trajectory 100**

traj_fmt

Key to specify the format of the trajectory file.

Options:

- 0 - Don't record trajectory
- 1 - Ymol xyz-style format
- 2 - XMOLE (xmakemol) xyz-style format
- 3 - DLPOLY (HISTORY) style format

Default: 1

Example: **traj_fmt 1**

vel_key

Key to either read in atomic velocities from the `.<Velocity>` box, or thermalise the system initially to dtemp.

Options:

- 0 - Generate velocities
- 1 - Read in velocities. Velocities must be provided in the `.<Velocity>` box (see section 8.7.2)

Default: 0

Example: `vel_key 0`

8.7.2 Velocity Box

Box Keyword: `.<Velocity>`

This box contains the initial velocities of all the free atoms in the system, and is only read in if **vel_key** in the MDcontrol box is set to 1, otherwise velocities are generated according to the given system temperature. The velocities given in this box must be in exactly the same order as the atomic coordinates in order to correspond to the correct atoms, if atoms have shells shell velocities can be given, otherwise the shell velocities are set equal to the atomic velocities. The units are Å/ps, and the format is as follows:

$v_{core_x} \ v_{core_y} \ v_{core_z} \ v_{shell_x} \ v_{shell_y} \ v_{shell_z}$

8.7.3 Friction Box

Box Keyword: `.<Friction>`

This box contains the friction coefficients for all the components of all the atoms in the system for the stochastic thermostat, and is only read in if the stochastic integration algorithm is selected in the MDcontrol box. This box must contain an entry for each atom in the system, in the order given in the atomic coordinate box. Each entry consists of an integer flag followed by the x, y and z components of the friction coefficient for that atom. If the flag is set to 0, the atom will not be connected to the stochastic thermostat, and its motion will be integrated using the standard Verlet leapfrog algorithm, if the flag is set to 1, the atom will be connected to the thermostat. E.g.:

1 $\gamma_x \ \gamma_y \ \gamma_z$

8.8 Dynamic Tip

Box Keyword: `.<TipDynamics>`

This option allows the frozen atoms of the tip to be moved during the MD or energy relaxation simulations. This option is implemented differently in either case.

8.8.1 During MD

A single string command is used to specify the tip movement during the scan:

howto $A \ T \ v_x \ v_y$

The tip can be scanned laterally at a constant velocity, or oscillated sinusoidally in the z-direction, or both at the same time. The option is followed by four parameters, as indicated above, which control the movement of the tip, according to the following equations of motion:

$$x = x_0 + v_x n \Delta t$$

$$y = y_0 + v_y n \Delta t$$

$$z = z_0 + A \sin\left(\frac{2\pi n \Delta t}{T}\right)$$

where n is the timestep number and Δt is the timestep itself. The units of A are Å, the units of T are ps, the units of v_x and v_y Å/ps.

Options: A T v_x v_y

Default: no

Example: **howto** 4.0 20.0 1.0 1.0

8.8.2 During energy relaxation

howto R_x R_y R_z N

where (R_x, R_y, R_z) is the step vector $\mathbf{R}$ (in Å) and N is the number of steps to be used to move the tip along the step direction $\mathbf{R}$.

turnback

If present, the tip will reverse its direction along the scan after performing N steps and should return to the initial position. Total number of points along the path is $2N$. This option is useful e.g. when studying adhesion effects.

8.9 Image Box

Box Keyword: .<Image>

If present, the polarisation of the electrodes (the image interaction) will also be applied. The options below within the Image box are used to produce an input file for the **image** code which is called from inside the **shellm**.

8.9.1 plane

This keyword includes the effect of a infinite conducting plane in the calculation of the image force.

Options: none

Default: off

Example: **plane**

8.9.2 Z_plane

This specifies the position of the infinite conducting plane on the z-axis in Å(#). It is only meaningful if the keyword **plane** is also present.

Options: #

Default: # = 0.0

Example: **Z_plane -15.0**

8.9.3 sphere

This keyword includes the effect of a conducting sphere in the calculation of the image force.

Options: none

Default: off

Example: **sphere**

8.9.4 bottom_sph

This specifies the position x , y , z of the bottom of the conducting sphere in cartesian coordinates. This is only meaningful if the keyword **sphere** is also present.

Options: x , y , z

Default: $x = 0.0$, $y = 0.0$, $z = 0.0$

Example: **bottom_sph 0.0 0.0 20.0**

8.9.5 radius

This specifies the radius of the conducting sphere in Å(#). This is only meaningful if the keyword **sphere** is also present.

Options: #

Default: # = 100.0

Example: **radius 50.0**

8.9.6 bias

This specifies the bias applied across the system in V (#).

Options: #

Default: # = 0.0

Example: **bias 1.5**

8.9.7 qwrite

This keyword controls whether the positions and charges of all image charges are outputted to the file *charges.out*.

Options: none

Default: off

Example: **qwrite**

8.9.8 printing

Controls amount of information in image force output files.

Options: **no, tiny, small, medium, large, huge**

Default: **tiny**

Example: **printing medium**

8.9.9 precis

Specifies the required tolerance in convergency of the image charge summation (#). If integer, it is the number of charges to be created (see section 1.4), otherwise it specifies the absolute tolerance.

Options: #

Default: **1.0e⁻⁶**

Example: **precis 50**

8.9.10 decay

This keyword specifies whether the charge of atoms near to the conducting sphere or plane is reduced as a function of distance. This is necessary when there are charges very near (< 2 Å) to a conductor, as the classical approximation breaks down at these distances and the image interaction is unrealistically large. The decay parameter removes this problem by reducing the magnitude of charges and therefore reducing the interaction.

Options: none

Default: off

Example: **decay**

8.9.11 smooth

This parameter controls the smoothness of the charge decay of atoms near the conductors (#). The bigger the value the smoother the decay, but the longer the range of decay.

Options: #

Default: # = 0.3

Example: **smooth 0.7**

8.9.12 buffer

This parameter controls the distance, in Å, from the conductors at which the charges on atoms begin to decay (#).

Options: #

Default: # = 1.8

Example: **buffer 5.0**

8.10 Constraints Box

Box Keyword: .<Constraints>

See the main points behind the constrained minimisations in section 1.7. Number of constrains is limited to NOCNSTR in `param.inc` file. Constraints are specified one per line.

- The format for linear constraints is:

$$\text{LINR } I_1 \ C_1 \ I_2 \ C_2 \ \dots$$

where I_1 , I_2 , etc. are coordinate indicators containing atomic number and a letter **x**, **y** or **z**, attached from the right. The coefficients C_1 , C_2 , etc. are associated with the corresponding atomic coordinates and corresponds to Eq. (28). For instance, the condition

$$\text{LINR } 11x \ 1.0 \ 12x \ -1.0 \ 9z \ 2.0$$

is equivalent to the following linear constraint:

$$x_{11} - x_{12} + 2z_9 = 0$$

Note that the first coordinate is used as the redundand one. In the example above this will be x_{11} .

- The format for the constraint in which the distance between two atoms in kept constant is:

$$\text{D_1x } N_1 \ N_2 \ d \pm$$

where **D_1x** is the keyword, N_1 and N_2 - atomic numbers, d - the distance (in Å) and $\pm$ is the sign (either + or -). The x coordinate of atom N_1 is considered as redundand.

- The format for the constraint in which the angle α between three atoms in kept constant is (to be fully implemented when necessary):

$$\text{C_1x } N_1 \ N_2 \ N_3 \ \alpha \pm$$

where **C_1x** is the keyword, N_1 , N_2 and N_3 - atomic numbers, α - the angle (in degrees) and $\pm$ is the sign (either + or -). The x coordinate of atom N_1 is considered as redundand.

- The format of the constraint in which the angle between two atoms and the x axis is kept, is given by:

$$\text{A_1x } N_1 \ N_2 \ \alpha \pm$$

where **A_1x** is the keyword, N_1 and N_2 - atomic numbers, α - the angle (in degrees) and $\pm$ is the sign (either + or -). The x coordinate of atom N_1 is considered as redundand.

8.11 Quantum cluster

The code can be run alongside the GAUSSIAN code in which case atoms in the system associated with the quantum cluster need to be identified. This option works presently only with pairwise interatomic interactions. To specify atoms belonging to the quantum cluster (QC) put the @ character immediately after the atomic symbol, e.g. :

Cr@ x y z (iRx iRy iRz)

If there are quantum cluster atoms in the system, the total energy will have a different meaning as the self-energy of the nvironment and its short-range interaction with the atoms of the QC.

This option was originally designed to calculate the electronic structure of the Cr^{3+} defect in MgO taking into account all polarisation charges in the junction as obtained by a non-selfconsistent run of the SCI-FI code.

Please, contact the present authors if interested in knowing this in more detail!

9 vAFM input

Options necessary to run vAFM and SciFi together, are marked with *.

9.1 Example Input File

This input file below runs altogether 29 stages of the virtual machine. The first two stages are needed to stabilise the cantilever, they are not shown in the box `<Elementary_Stages>` since are run anyway (default). The 3rd stage corresponds to the approach of the tip to the surface to get the -440 Hz frequency shift. After this stage the tip scans along the Y direction for 0.02 s. Then a 2D scan starts which consists of 3 sequences of 4 elementary line scans (passages) each, 12 elementary line scans altogether. The scan is performed keeping the same frequency shift and amplitude. This first 2D scan is followed by a single stage in which the tip is not moved laterally, but the amplitude is reduced by 10 Å. Finally, the second scan is performed, identical to the first one with the original amplitude and frequency shift.

```
....<VirtualAFM>
Q-factor 10000.0d0
Elastic_Constant 6.0
Natural_Frequency 60406.0
SetPnt_Amplitude 5.0e-9
SetPnt_FreqShift -100.0
Stabil_Time1 4
Stabil_Time2 80.d-3
PLL_FMdem{t1,t2,gainPB,K1} 5.5d-5 1.6d-4 6.d0 500.d0
AGC{P,I,tau} 0.1 50.0 450.d-6
PLL_ADC{P,I0,I} 1.d-13 1.d-9 1.d-9
Debug F 100
noise{F,A,Temp} F F 300.0
Print_Cycles 2
Start{X,Y} 0.0d-10 0.0d-10
.....<End>

....<Force_Field>
Force_field_file 1.dat
Pixels{1,2,z} 1 79 44
vdW{H,R} 0.99864 300.0
Long-Range{bound,z0,g0,...,g6} 16.0 -5.0928 0.0 0.0 0.0 -0.40916 0.0 0.0 0.0
Lattice{/A1(xy),/A2(xy)} 1.0 0.0 0.0 16.1 Force_Test T 0.1
.....<End>

....<Elementary_Stages>
_{begin_stage}_ <approach>
Finish{how,time} freq
Velocity{Vx,Vy} 0.0 0.0
Prt{T/F,how_often} T 100
ADC{T/F,freq} T -440.0
_{begin_stage}_
Finish{how,time} time 0.02
Velocity{Vx,Vy} 0.0 1.0e-9
Prt{T/F,how_often} T 100
ADC{T/F,freq} T -440.0
==={begin_scan}===
No_of_sequences 3
_{begin_stage}_
Finish{how,time} time 0.01
Velocity{Vx,Vy} 0.0 1.0e-9
ADC{T/F,freq} T -440.0
_{begin_stage}_
Finish{how,time} time 0.001
Velocity{Vx,Vy} 1.0e-9 0.0
ADC{T/F,freq} T -440.0
_{begin_stage}_
Finish{how,time} time 0.01
Velocity{Vx,Vy} 0.0 -1.0e-9
ADC{T/F,freq} T -440.0
_{begin_stage}_
Finish{how,time} time 0.001
Velocity{Vx,Vy} 1.0e-9 0.0
ADC{T/F,freq} T -440.0
==={end_scan}===
```

```

_{begin_stage}_
Finish{how,time} amplt
Velocity{Vx,Vy} 0.0 0.0
ADC{T/F,freq} T -440
AGC{T/F,A0} T 0.4e-8
==={begin_scan}===
No_of_sequences 3
_{begin_stage}_
Finish{how,time} time 0.01
Velocity{Vx,Vy} 0.0 1.0e-9
ADC{T/F,freq} T -440.0
_{begin_stage}_
Finish{how,time} time 0.001
Velocity{Vx,Vy} 1.0e-9 0.0
ADC{T/F,freq} T -440.0
_{begin_stage}_
Finish{how,time} time 0.01
Velocity{Vx,Vy} 0.0 -1.0e-9
ADC{T/F,freq} T -440.0
_{begin_stage}_
Finish{how,time} time 0.001
Velocity{Vx,Vy} 1.0e-9 0.0
ADC{T/F,freq} T -440.0
==={end_scan}===
.....<End>

```

Three boxes are to be supplied to run the code which are described below.

Note that the box above corresponds to the vAFM run without SciFi. If, however, it is included into the SciFi input file and options necessary to run the two codes together are also specified, then both codes will be working at the same time.

9.2 Virtual AFM Box

Box Keyword: .<VirtualAFM>

This box contains all the main parameters of the cantilever, default values for the electronic systems (ADC, AGC, PLL), including variables controlling the amount of the output produced by the code. Any input in the box is given using a keyword followed by one or more parameters. Every such entry can be positioned independently within the box. The order of the parameters after the keyword is fixed; in most cases it is hinted in the keyword itself.

9.2.1 General settings for the cantilever

All correspond to the LHS of Eq. (30).

Elastic_Constant

The value of the cantilever elastic constant k (in N/m).

Parameters: k

Default: 6.0 N/m

Example: Elastic_Constant 6.5

Natural_Frequency

Natural frequency f_0 of the cantilever far away from the surface, in Hz.

Parameters: f_0

Default: 60406.0 Hz

Example: Natural_Frequency 6.0500e5

Q-factor

Q factor (dimensionless) of the cantilever, Q .

Parameters: Q

Default: 10000.0

Example: `Q-factor 10000.0d0`

9.2.2 Default set points

Every stage of the virtual AFM is characterised by a number of set points. If they do not change during the experiment, it is convenient to input them here in this box as default variables.

SetPnt_Amplitude

The value for the oscillation amplitude (in m) for the AGC, A_0 .

Parameters: A_0

Default: 31.25d-9 m

Example: `SetPnt_Amplitude 5.0e-9`

SetPnt_FreqShift

The value for the frequency shift, negative (in Hz), for the ADC, Δf_0 .

Parameters: Δf_0

Default: -100.0 Hz

Example: `SetPnt_FreqShift -100`

noise{F,A,Temp}

This entry regulates the existence of the noise for the frequency and the amplitude, and is also used to input the temperature T (needed to calculate the variances for the two noises).

Parameters: N_F, N_A, T

Default: F, F, 300 K

Example: `noise{F,A,Temp} F T 350.0`

Here N_F is either T (for .true.) or F (.false.), it switches on or off the noise for the frequency, while N_A similarly controls the amplitude noise.

Debug

A detailed printout every N timesteps is controlled by this entry for each stage. The printout is given into a file `stageM.out`, where M is the stage number.

Parameters: N_D, N

Default: F, 10

Example: `Debug T 15`

9.2.3 Initial position of the tip

Start{X,Y}

Sets initial coordinates X_0 and Y_0 for the tip (with respect to the force field file). Zero values correspond to the first point (0,0) of the force field file.

Parameters: X_0, Y_0

Default: 0.0 0.0 (in m)

Example: Start{X,Y} 1.0d-10 -1.0d-10

Initial_Z

Initial value of the vertical position of the tip (far from the surface), D_1 (in m), Eq. (40).

Parameters: D_1

Default: 34.25d-09 m

Example: Initial_Z 40.0d-09

9.2.4 The frequency of the general printout

Print_Cycles

Presently, the printout to the file `signals.dat` is provided at the end of each N -th oscillation cycle.

Parameters: N

Default: 100

Example: Print_Cycles 10

9.2.5 Parameters for the Automatic Gain Control (AGC)

AGC{P,I,tau}

Parameters of AGC: K_P^{AGC} (in N/m), K_I^{AGC} (in N/m*s) and τ_D (in s) as in Eqs. (31) and (32), section 2.1.2.

Parameters: $K_P^{AGC}, K_I^{AGC}, \tau_D$

Default: 0.1, 2.0, 0.002

Example: AGC{P,I,tau} 0.1 50.0 450.d-6

9.2.6 Parameters for the Phase Lock Loop (PLL)

PLL_FMdem{t1,t2,gainPB,K1}

Parameters of PLL: τ_1, τ_2 (both in s), gain factors G and K_1 (see Eqs. (33)-(37) in section 2.1.3).

Parameters: τ_1, τ_2, G, K_1

Default: 5.0d-5, 1.6d-4, 6.0, 220.0

Example: PLL_FMdem{t1,t2,gainPB,K1} 5.5d-5 1.6d-4 6.d0 500.d0

9.2.7 Parameters for the Automatic Distance Control (ADC)

PLL_ADC{P,I0,I}

Parameters for the ADC: K_P^{ADC} (in m*s), initial $K_{I_0}^{ADC}$ (in m, only for approach at stage 3), working $K_{I_1}^{ADC}$ (in m, for any other stage) as in Eq. (40) of section 2.1.4.

Parameters: K_P^{ADC} , $K_{I_0}^{ADC}$, $K_{I_1}^{ADC}$

Default: 7.0d-11, 50.d-9, 5.d-9

Example: PLL_ADC{P,I0,I} 1.d-13 1.d-9 1.d-9

9.2.8 Times

A number of time-related parameters is specified using several keywords:

t-steps_in_period

Number of timesteps per oscillation period, N_{osc} ; is used to calculate the timesteps itself using $\Delta t = 1/(f_0 N_{osc})$.

Parameters: N_{osc}

Default: 400

Example: t-steps_in_period 500

Stabil_Time1

Duration time (in numbers of oscillation periods) of the first stabilisation stage, n_1 , when excitation is still switched off.

Parameters: n_1

Default: 4

Example: Stabil_Time1 4

Stabil_Time2

Duration time (in s) of the second stabilisation period, t_2 , when excitation is on, but the tip is still far away from the surface.

Parameters: t_2

Default: 15.0d-3

Example: Stabil_Time2 15.0d-3

9.3 Force Field Box

Box Keyword: .<Force_Field>

This box contains all the necessary information necessary to specify the force field for the tip-surface interaction. The force field consists of three components: the van der Waals force, chemical short-range and long-range forces.

Presently, the vdW force is given within the plane-sphere model,

$$F_{vdW}(z) = -\frac{HR}{6z^2} \quad (45)$$

where H is the Hamaker's constant, R - the sphere radius, z - distance of the sphere bottom from the surface plane.

The long-range chemical component of the force, which works above some critical vertical height z_{bound} , is generally given by the expression:

$$F_{lr}(z) = \sum_{n=0}^6 \frac{g_n}{(z + z_0)^n}, \quad z \geq z_{bound} \quad (46)$$

where g_n and z_0 are fitting parameters.

The short-range part of the chemical force is specified in a file on a lateral grid with respect to the direct lattice vectors of the surface, $\mathbf{a}_1$ and $\mathbf{a}_2$. At each grid point the force is specified for a number of N_z (not necessarily equidistant) z values below z_{bound} . If the grid points along $\mathbf{a}_1$ are numbered by $0, 1, \dots, N_1$ and similarly for the points along the $\mathbf{a}_2$ direction, numbered as $0, 1, \dots, N_2$, then the force is provided for every i, j ($i = 0, \dots, N_1$ and $j = 0, \dots, N_2$). To preserve periodicity of the lattice, the force fields for $i = 0$ and $i = N_1$ (any j) should be identical; similarly for values of $j = 0$ and $j = N_2$ (any i). The format of the force field file is described in section 9.3.5 below.

Two examples of the force field for the MgO (001) surface are given in Fig. 12. In the directory `exam-`

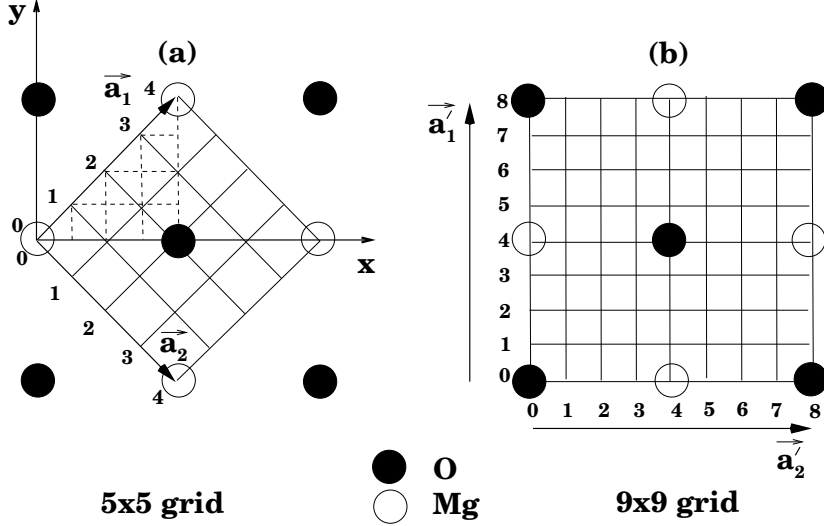


Figure 12: How the force field can be specified for the MgO (001) surface: (a) in the case of the 5x5 grid, the lattice vectors $\mathbf{a}_1$ and $\mathbf{a}_2$ correspond to the primitive unit cell; (b) in the case of the 9x9 grid, a two times larger cell is chosen with the lattice vectors $\mathbf{a}'_1 = \mathbf{a}_1 - \mathbf{a}_2$ and $\mathbf{a}'_2 = \mathbf{a}_1 + \mathbf{a}_2$. The original grid used to calculate the force field in non-equivalent points in the primitive cell is shown by dashed lines in (a).

ples/MgO/force_field you can also find raw data for the MgO force field calculated using the Sci-Fi code. The calculations were performed using a 64 atom 4x4x4 MgO cube tip with a Mg atom at the tip apex. The raw data for the force field are set in directories

```
\X=0,Y=0,
  \X=0.5305,Y=0.5305,
  \X=0,Y=2.122,
```

etc., corresponding to different values of the X and Y within the primitive cell as shown by the dashed lines in Fig. 12 (a). The coordinate axes are those shown in Fig. 12, and the Mg-O distance is equal to 2.122 Å. The results of the calculation for a 2D scan using either of the force field are collected in directories `examples/MgO/results_5x5` and `examples/MgO/results_9x9` (see also section 11.4).

The information in the box is provided in the same way as in the previous case: each time a keyword is used which is followed by the necessary parameters. The order of each input is arbitrary, the order within each input is fixed.

Important: contrary to all other boxes, all values here are provided using eV and Å units!!!

9.3.1 How to specify the short-range force

Force_field_file

Name of the file containing the short-range component of the force field. Must only be supplied if the vAFM is to be run without SciFi.

Parameters: filename (up to 50 characters)

Default: empty name

Example: Force_field_file CaCO3.dat

Pixels{1,2,z}

Number of grid points N_1 and N_2 along directions $\mathbf{a}_1$ and $\mathbf{a}_2$, respectively, followed by the number N_z of points along z .

Parameters: N_1, N_2, N_z

Default: 1,1,44

Example: Pixels{1,2,z} 25 49 44

Lattice{/A1(xy),/A2(xy)}

Components along X and Y axes of the laboratory Cartesian system of the lattice vectors $\mathbf{a}_1$ and $\mathbf{a}_2$ are given (in Å) in two lines coming next after the keyword: a_{1x}, a_{1y} on the first and a_{2x}, a_{2y} on the second.

Default: none

Example: Lattice{/A1(xy),/A2(xy)}
1.0 0.0
0.0 1.0

9.3.2 Long-range part of the force

Long-Range{bound,z0,g0,...,g6}

The parameters of the long-range part of the chemical force, Eq. (46), are given using the Å/eV units, so that the force would be in eV/Å and distances in Å.

Parameters: $z_{bound}, z_0, g_0, g_1, \dots, g_6$

Default: 16.0, -5.0928, 0.0, 0.0, 0.0, -0.40916, 0.0, 0.0, 0.0

Example: Long-Range{bound,z0,g0,...,g6} 16.0 -5.0928 0.0 0.0 0.0 -0.40916 0.0 0.0 0.0

vdW{H,R}

Parameters for the vdW force (the force is in eV/Å) as in Eq. (45).

Parameters: H, R

Default: 0.99864, 300.0

Example: vdW{H,R} 0.99864 300.0

9.3.3 *Parameters necessary to run vAFM and SciFi together

Atom_Sphere{#,dist}

Parameters for placing the vdW sphere within the atomistic part; only needed if vAFM and SciFi are to work together. Should specify two parameters: (i) atom number which is used to measure the distance to the vdW sphere, I , and the distance D (in Å) to the bottom of the vdW sphere from this atom.

Parameters: I, D

Default: 1, 0.0

Example: Atom_Sphere{#,dist 15 1.25

interpolate{F/T,points}

Two parameters are supplied: a logical (False or True) and the number of points N . If the first variable is False, then no interpolation of the tip force is done, it will be calculated at each tip position by running full SciFi ionic relaxation calculation. If, however, the first parameter is True, then an interpolation of the force between two nearest known values (for vertical positions of the tip only) will be done, atomic relaxation will not be performed saving computer time.

The parameter N specifies the number of nearest forces to be used in the interpolation (only $N = 2$ is presently implemented).

Parameters: F, N or T, N

Default: T,2

Example: Force_Test 0.01

extrapolate{F/T,polyn,points}

This option is supposed to control extrapolation of the tip force - not yet implemented. The first parameter is F (do not apply) or T (do apply extrapolation mechanism to the tip force), then the order of a polynomial N_1 to be used for the extrapolation, and , finally, the number of points N_2 .

Parameters: F, N_1, N_2 or T, N_1, N_2

Default: T,2,3

Example: Fextrapolate{F/T,polyn,points} T 2 3

smooth{F/T,points}

Controls smoothing out the forces using a certain number of points, N_s .

Parameters: F, N_s or T, N_s

Default: T,2

Example: smooth{F/T,points} T 2

9.3.4 Testing the whole range force field

Force_Test

The force field can be tested prior to (and instead of) any calculation if $N_{ft}=T$ is used as the first parameter; the second parameter in this case is a step, Δz , along the z direction (in Å).

Parameters: $N_{ft}, \Delta z$

Default: F, 0.01

Example: Force_Test 0.01

9.3.5 Structure of the force-field file

Only relevant if the force field file is specified, i.e. if the SciFi part is switched off.

The short-range part of the force field is specified in a file as was mentioned in section 9.3.1 above. Let us assume that the lateral grid is N_1 , N_2 and the number of points along the Z axis is N_z . Then, the format of this file is as follows:

- the 1st line: comment
- N_z lines follow with the z values (in the ascending order)
- a comment line
- then the force field for every i, j along the lateral grid mesh follows in blocks as given by the following FORTRAN statement:

```
do i=0,N1

do j=0,N2

.....

end do

end do
```

- each block contains the values of the force for all values of z (in the same order as the values of z themselves, of course!), and it starts with a comment line.

9.4 Elementary Stages Box

Box Keyword: .<Elementary_Stages>

9.4.1 What controls every elementary stage?

Parameters controlling every elementary stage are listed below:

- amplitude noise ON/OFF (logical);
- force noise ON/OFF (logical)
- ADC is ON/OFF (logical):
 - if ON, then the z distance is adjusted so that the frequency shift Δf could be maintained approximately constant close to the set value, Δf (real);
 - if OFF, then the distance is kept exactly constant instead (constant height mode); the frequency will change along the simulation, still kept at resonance;
- AGC is ON/OFF (logical):
 - if ON, then gain control allows to keep the oscillation amplitude approximately constant close to the set value, A (real), by constantly adjusting the excitation amplitude;
 - if OFF, then the excitation amplitude is kept exactly constant, whereas the oscillation amplitude may change along the simulation;
- velocities v_x and v_y along the X and Y axes, kept constant during every elementary stage (two real);
- a logical variable which switches ON/OFF the tip-surface force;
- a logical variable which switches ON/OFF the excitation signal applied to the cantilever;

- a logical variable which switches ON/OFF the detailed printout during each stage to be made into the file `stageM.out` (M being the stage number); if ON, then there is also an integer variable equal to the number of timesteps after which the printout will be made;
- finally, presently there are three ways in which each stage can be terminated (a character variable):
 - `'time'` followed by the time in seconds;
 - `'freq'`: the stage will run to meet the frequency shift setpoint for this stage (up to an error of 0.5 Hz);
 - `'ampl'`: the stage will run to meet the oscillation amplitude setpoint for this stage (up to an error of 0.1 Å).

9.4.2 Specifying each elementary stage

Each elementary stage within the box is specified in a sub-box which is started by a keyword:

`_ {begin_stage}_`

followed by the “keyword + parameters” structure identical to other boxes; the sub-box ends if either the next `_ {begin_stage}_` keyword is found (to start the next stage sub-box) or the

`.<End>`

marker of the entire box is found. All the parameters which control the execution of every stage are initially set to the default values from the `<VirtualAFM>` box of section 9.2. Therefore, it is only necessary to give for each stage all those parameters which are different from the default values. All the keywords and their corresponding parameters are listed below.

Finish{how,time}

Three methods to finish the stage (as above).

Options: `'time'`, `'freq'`, `'ampl'`

Parameters: time t (in s) when option `'time'` is used

Default: `'time'` with 0.0 s

Examples: `Finish{how,time} time 0.01`
`Finish{how,time} freq`
`Finish{how,time} ampl`

Excite{T/F}

Switches ON/OFF the excitation signal applied to the cantilever.

Parameters: F/T

Default: T

Example: `Excite{T/F} T`

ADC{T/F,freq}

Switches ON/OFF the ADC; if ON, then the set point frequency shift (negative) is also provided (in Hz).

Parameters: F/T, Δf

Default: T, Δf_0

Example: `ADC{T/F,freq} T -100.0`

AGC{T/F,A0}

Switches ON/OFF the AGC; if ON, then the set point amplitude A is also provided (in m).

Parameters: F/T, A

Default: T, A_0

Example: AGC{T/F,A0} T 0.1e-9

Noise{F,A}

Switches ON/OFF the noise for the force and the amplitude (in this order).

Parameters: F/T, F/T

Default: as in <VirtualAFM> box

Example: Noise{F,A} T T

Prt{T/F,how_often}

Switches the detailed printing ON/OFF; if ON, then it prints a number of essential variables along the simulation into the file `stageM.out` (M being the stage number) every M_{dt} timesteps.

Parameters: F/T, M_{dt}

Default: as in <VirtualAFM> box

Example: Prt{T/F,how_often} T 10

Velocity{Vx,Vy}

Sets velocities v_x and v_y along the X and Y directions (in m/s) for the given stage.

Parameters: v_x , v_y

Default: 0.0, 0.0

Example: Velocity{Vx,Vy} 0.0e0 1.0e-9

Tip_Force{T/F}

Switches ON/OFF the tip-surface force.

Parameters: T/F

Default: T

Example: Tip_Force{T/F} T

9.4.3 Doing a 2D scan

Using elementary stages as described above one can program in any particular movement of the cantilever across the surface including doing a 2D scan. This last option may be tedious though since many times the same pattern of line scans should be repeated. To avoid this, the 2D scan can be automated using another sub-box structure inside the <Elementary_Stages> box.

The basic idea is that a 2D scan can be simulated by repeating e.g. 4 elementary stages:

1. a line scan along direction $\mathbf{b}_1$ during a long time t_1
2. a line scan along the perpendicular direction $\mathbf{b}_2$ during a very short time t_2
3. a line scan along direction $-\mathbf{b}_1$ during time t_1

4. a line scan along the perpendicular direction $\mathbf{b}_2$ during a very short time t_2

Some other possibilities for a 2D scan also exist, e.g. by repeating only 3 stages:

1. a line scan along direction $\mathbf{b}_1$ during a long time t_1
2. a line scan along direction $-\mathbf{b}_1$ during the same time t_1
3. a line scan along the perpendicular direction $\mathbf{b}_2$ during a very short time t_2

Thus any scan can be viewed as an identical sequence of structures, each consisting of several elementary stages. The number of sequences determines the length of the scan in the direction $\mathbf{b}_2$, whereas the length of the scan in the perpendicular direction $\mathbf{b}_1$ is determined by the time t_1 in the examples above.

It is possible to position the scan sub-box arbitrarily between any stage sub-boxes inside the `<Elementary_Stages>` box as shown in the example in section 9.1. Therefore, it is possible to do some number of elementary stages, then a scan, then some other elementary stages, another scan, etc. The total length of this chain of stages is only limited by the maximum number of stages and scans allowed as given by the two parameters in the `virtual_param.inc` file (see section 5). Note that every scan is eventually constructed as a set of stages, so that the parameter `N_of_stages_max` can easily become too small and need to be increased, and the code recompiled.

To start a scan, place the keyword

```
==={begin_scan}===
```

after a stage sub-box². The next line *should* contain the “keyword + parameter” structure:

No_of_sequences

Parameters: number of sequences

Default: none

Example: No_of_sequences 100

This parameter defines the number of times each sequence is to be repeated. After that, the elementary sequence, consisting of elementary passages (i.e. stages) follows in the same way as described in section 9.4.2. The scan sub-box is terminated with the keyword

```
==={end_scan}===
```

For instance, the 4-stage/passage scan structure to be repeated 10 times along the X direction is given by the following scan sub-box:

```
==={begin_scan}===
No_of_sequences 10
_{begin_stage}_
Finish{how,time} time 0.01
Velocity{Vx,Vy} 0.0 1.0e-9
ADC{T/F,freq} T -440.0
_{begin_stage}_
Finish{how,time} time 0.001
Velocity{Vx,Vy} 1.0e-9 0.0
ADC{T/F,freq} T -440.0
_{begin_stage}_
Finish{how,time} time 0.01
Velocity{Vx,Vy} 0.0 -1.0e-9
ADC{T/F,freq} T -440.0
_{begin_stage}_
Finish{how,time} time 0.001
Velocity{Vx,Vy} 1.0e-9 0.0
ADC{T/F,freq} T -440.0
==={end_scan}===
```

Here Δf is set to -440 Hz during the whole scan.

²Note that you have to do the approach anyway, so that the stage sub-box should always precede any of the scan boxes.

Part IV

Output

10 SciFi specific output

10.1 Standard Output

The output to the screen will tell you the main parameters for your calculation and will also warn you of any obvious inconsistencies that the program is trying to compensate for. If there are any major problems with the input the program will exit with a fatal error and a message explaining the nature of the error.

10.2 Output File

Every run will produce an output file called **name.OUT**, this will contain detailed information (depending on the printing parameter) about your system and the calculations run on it. The first stage of the output file will give all the input parameters as recognized by the program followed by some basic information about your system derived from those parameters. This is the first place to check for consistency, make sure that the output is consistent with the system you intended in your input. The rest of the output file will detail the process of the calculations, including a display of the total energy and all forces (and the source of those forces) on the atoms at each relaxation iteration. If an energy minimisation calculation is being performed, then at the end of the file the total force on the tip (if there was one) will be displayed along with a total force breakdown.

10.3 Configuration Files

If a minimisation calculation is being performed and the **xyz** option is present in the Toleranc box, then the program will produce two **.xyz** format files: **name_core.xyz** and **name_shell.xyz**, containing the final relaxed coordinates of the cores and shells in the system correspondingly. If a scan is performed using the energy minimisation method (i.e. the TipDynamics box is present), the two files will contain coordinates of cores and shells at every point at the tip path one after another.

10.4 Statistics File

If a Molecular Dynamics simulation is being performed, the run will produce a file with thermodynamic data at intervals set with the **statistics** variable, with the filename **name.STS**. This file consists of a line for every interval, split into six columns that give: timestep, total system energy, potential energy, kinetic energy, temperature and shell temperature.

The same file is also produced during a scan made using options in the TipDynamics box. If only energy minimisation is performed (the box MDcontrol is absent) and the tip is allowed to move (the TipDynamics box is present), the file will contain only total energies of the system at every scan point.

10.5 Trajectory File

An MD run will produce a trajectory file named **name.TRJ** of the format and interval specified in the MDcontrol box. This file contains the atomic coordinates of the system at each interval.

10.6 Tip Force Files

If a Molecular Dynamics simulation is being performed, then the run will produce a file with the total z-component of the force on the tip at every timestep called **name.TFZ**. If the **do_lat** flag is used in the MDcontrol box, then x- and y-components of the total force on the tip at every timestep will be written to the files **name.TFX** and **name.TFY** respectively.

The same files are produced during a scan (i.e. when the TipDynamics box is present) if only energy minimisation is performed (the box MDcontrol is absent).

10.7 Output from image

If the image input box is activated correctly then the image interaction in your system will be calculated and several other output files will be produced. In general it is not necessary to look at these, but they are useful for detailed analysis of the image force and also for finding the source of any strange behaviour in the calculations. The possible files produced are as follows:

- *charges.out* - which contains the position (in a.u.) and charge of every image charge created (if **qwrite** is on).
- *forces.out* - which contains the image force on every atom (excluding the **frozen-tip** atoms) and the total image force.
- *image.out* - which contains an output of the parameters used in calculating the image force.
- *out* - standard output from calculating image force on atoms.
- *outf* - standard output from calculating the image force on the tip.
- *potent.out* - which contains the image potential on all atoms in the system and the image energy.

11 vAFM specific output

The output to the screen will tell you the main parameters for your calculation and will also show the processing of each elementary stage. In addition, a number of output files are produced.

11.1 virtual.out

This file starts with detailed information concerning with the current calculation. In particular, it contains a detailed account of all the stages made of both individual stages and 2D scans, in a form of a table. After the input part, it gives some useful information about each stage in the process of the calculation, specifically about the setpoints and the initial values of some variables. An example of this file is given in the directory `/examples` for the input file of section 9.1.

11.2 stageM.out

This type of file with M being the stage number is created at every stage provided that the detailed printing is ON. The file starts from the heading which explains the variables it contains:

```
# excite - excitation signal at timestep t
# DEF - frequency shift (Hz)
# ds - distance of closest approach (A)
# dist - actual distance at time t (A)
# ftip - force (eV/A)
# A - oscillation amplitude
#
# t          z(t)      excite    DEF      ds      ftip      dist      A
```

Note that variables printed out during the first two stages are slightly different.

11.3 signals.dat

This single file contains the output (in the correct order of stages) in the following form:

```
# oscil - oscillation number
# X,Y - lateral position (in A)
# A - oscillation amplitude (in A)
# DEF - frequency shift (Hz)
# Ediss - dissipation energy per cycle (eV)
# appr - distance of closest approach (A)
#
# oscil    X      Y      A      DEF      Ediss      appr
```

Variables printed into the file are calculated every N -th oscillation cycle at the end of the oscillation period (when the tip is at the top most position). Here N is the parameter specified after the keyword **Print_Cycles**, section 9.2.4.

11.4 2Dimage_S.dat

For each scan a file with this name is produced (S is the scan number), which contains:

```
# X,Y - lateral position (in A)
# A - oscillation amplitude (in A)
# DEF - frequency shift (Hz)
# Ediss - dissipation energy per cycle (eV)
# appr - distance of closest approach (A)
#
#      X      Y      A      DEF      Ediss      appr
```

Difference with the `signal.dat` file is two-fold: firstly, the oscillation number is not printed; secondly, different files are produced for each stage.

As an example, we show in Fig. 13 the 2D topology image of the MgO (001) surface calculated by the code using the 9x9 force field (section 9.3). Black spots correspond to Mg atoms, while white ones - to O atoms. The force

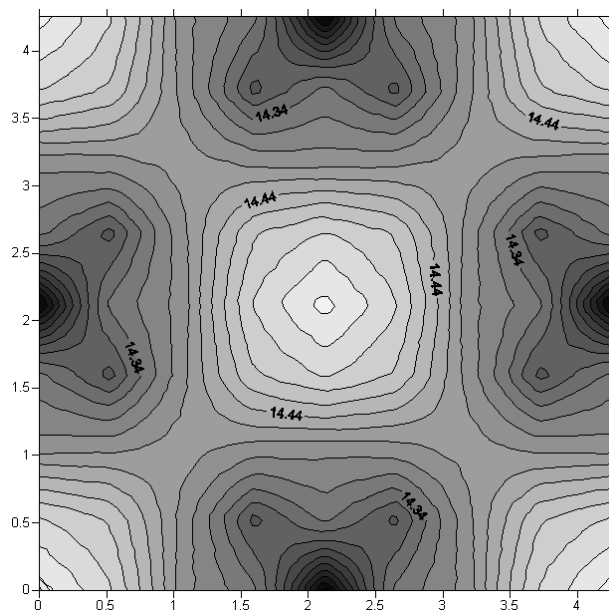


Figure 13: 2D topography image of the MgO (001) surface calculate using the 9x9 grid force field.

field was obtained using a 64 atom 4x4x4 MgO cube with a Mg atom at the tip apex. This is in perfect qualitative agreement with the image shown.

11.5 ftip.dat

This file is produced if the tip force is tested (section 9.3.4). Its heading reflects its contents:

```
# force field for given X,Y=( 0.00000, 0.00000)
#      z (in A)      vdW (in eV/A)      chemical (in eV)      total (in eV/A)
```

Part V

Example Inputs

A large number of input files are contained in the subdirectory **examples** together with corresponding output files. The directory is broken down into three subdirectories corresponding to

1. SciFi only runs
2. vAFM only runs
3. SciFi+vAFM runs: in this case only approach has been modelled, i.e. the same lateral position of the tip has been used

Most of the examples are based on an insulating MgO surface interacting with a model 64 atom cube MgO tip, e.g.

- mgo.INN - MD simulations of an insulating MgO tip interacting with a MgO sample
- mgo1.INN - test of the tip force for an insulating tip interacting with cluster
- mgo_mt.INN - lateral scan of an MgO tip above the MgO sample.

References

- [1] L. N. Kantorovich, A. I. Livshits, and A. M. Stoneham. Electrostatic energy calculation for the interpretation of surface probe microscopy experiments. *J. Phys.: Condens. Matter*, 12:795–814, 2000.
- [2] L. N. Kantorovich, A. Foster, A. L. Shluger, and A. M. Stoneham. Role of image forces in nc-sfm images of ionic surfaces. *Surface Sci.*, 445:283, 2000.
- [3] J. A. Stratton. *Electromagnetic Theory*. McGraw-Hill, 1941.
- [4] L. D. Landau, E. M. Lifshitz, and L. P. Pitaevskii. *Electrodynamics Of Continuous Media*. Pergamon Press, Oxford, 1993.
- [5] J.D. Jackson. *Classical Electrodynamics*. John Wiley, N. Y., 1999.
- [6] M. W. Finnis. The interaction of a point charge with an aluminium (111) surface. *Surface Sci.*, 241:61, 1991.
- [7] M. W. Finnis, R. Kaschner, C. Kruse, J. Furthmüller, and M. Scheffler. The interaction of a point charge with a metal surface: theory and calculations for (111), (100) and (110) aluminium surfaces. *J. Phys.: Condens. Matter*, 7:2001, 1995.
- [8] M. García-Hernández, P. S. Bagus, and F. Illas. A new analysis of image charge theory. *Surface Sci.*, 409:69, 1998.
- [9] W. R. Smythe. *Static and Dynamic Electricity*. McGraw-Hill, N. Y., 1968.
- [10] C. Argento and R. H. French. Parametric tip model and force-distance relation for hamaker constant determination from atomic force microscopy. *J. Appl. Phys.*, 80:6081, 1996.
- [11] L. N. Kantorovich. Quantum theory of energy dissipation in non-contact atomic force microscopy in markovian approximation. *Surface Science*, 521:117–128, 2002.
- [12] M. Gauthier, L. Kantorovich, and M. Tsukada. *Non-Contact Atomic Force Microscopy*, chapter 19. Theory of energy dissipation into surface vibrations. Nanoscience and Technology. Springer, 2002.
- [13] T. Trevethan and L. Kantorovich. Tip models and force definitions in molecular dynamics simulations of scanning force microscopy. *Surface Science*, 540:497–503, 2003.
- [14] J. Tersoff. New empirical approach for the structure and energy of covalent systems. *Phys. Rev. B*, 37:6991, 1988.

- [15] B. G. Dick and A. W. Overhauser. *Phys. Rev.*, 112:603, 1958.
- [16] M. Allen and D. Tildesley. *Computer simulation of liquids*. Oxford Press, 1989.
- [17] J. Clarke D. Brown. *Mol. Phys.*, 51:1243, 1983.
- [18] W. Hoover. *Phys. Rev. A*, 31:1695, 1985.
- [19] J. Rullmann W. Gunsteren, H. Berendsen. Stochastic dynamics for molecules with constraints, brownian dynamics of n-alkanes. *Mol. Phys.*, 44:69, 1981.
- [20] D. Fincham P. Mitchell. Shell model simulations by adiabatic dynamics. *J. Phys. Cond. Matt.*, 5:1031, 1993.
- [21] M. Gillan P. Lidan. Shell-model molecular dynamics simulations of superionic conduction in CaF_2 . *J. Phys. Cond. Matt.*, 5:1019, 1993.
- [22] *Omicron Nanotechnology*.
- [23] *easyPLL Nanosurf*.
- [24] U. Dürig, H. R. Steinauer, and N. Blanc. ? *J. Appl. Phys.*, 82:3841–3651, 1997.
- [25] Ch. Loppacher, M. Bammerlin, M. Guggisberg, F. Battiston, R. Bennewitz, S. Rast, A. Baratoff, E. Meyer, and H.-J. Güntherodt. ? *Appl. Surf. Sci.*, 140:287–292, 1999.
- [26] O. Pfeiffer, L. Nony, R. Bennewitz, A. Baratoff, and E. Meyer. ? *Nanotechnology*, pages S101–S107, 2004.
- [27] L. Verlet. ? *Phys. Rev.*, 159(1):98–103, 1967.
- [28] M. Etique. *Regulation Automatique*. Ecole d’Ingénieurs du Canton de Vaud, Switzerland, 2003.
- [29] G. J. Silva, A. Datta, and S. P. Bhattacharyya. ? *Automatica*, 37:2025–2031, 2001.
- [30] Y.-G. Wang and H.-H. Shao. ? *Automatica*, 36:147–152, 2000.
- [31] Ch. Loppacher, M. Bammerlin, F. Battiston, M. Guggisberg, D. Müller, H. R. Hidber, E. Lüthi, R. and Meyer, and H.-J. Güntherodt. ? *Appl. Phys. A*, 66:S215–S218, 1998.
- [32] Ch. Loppacher. *Nichtkontaktrasterkraftmikroskopie Mit Digitalem Phasenregelkreis*, PhD. Thesis. PhD thesis, Basel University, Basel, 2000.
- [33] T. R. Albrecht, P. Grütter, D. Horne, and D. Rugar. ? *J. Appl. Phys*, 69(2), 1991.
- [34] U. Dürig, O. Züger, and A. Stalder. ? *J. Appl. Phys.*, 72(5):1778–1798, 1992.
- [35] D. Sarid. *Scanning Force Microscopy*. Oxford University Press, Oxford, 1991.
- [36] G. G. Yaralioglu, A. Atalar, S. R. Manalis, and C. F. Quate. ? *J. Appl. Phys.*, 83:7405–7415, 1998.
- [37] A. N. Cleland and N. L. Roukes. ? *J. Appl. Phys.*, 92:2758–2769, 2002.
- [38] A. M. Bacon, H. Z. Zhao, L. J. Wang, and J. E. Thomas. ? *Appl. Optics*, 34(24):5326–5330, 1995.
- [39] M. Guggisberg, M. Bammerlin, Ch. Loppacher, O. Pfeiffer, A. Abdurixit, W. Barwich, R. Bennewitz, A. Baratoff, E. Meyer, and H.-J. Güntherodt. ? *Phys. Rev. B*, 61:11151–11155, 2000.